SỔ TAY THỰC HÀNH

React JS
dễ hiểu

20 bài học bằng tiếng Việt

Theo thứ tự các chủ đề trong video Bro Code

Lý thuyết ngắn gọn • Code hoàn chỉnh • Làm từng bước

JavaScript + Vite • Cách viết phù hợp React 19

```
function HocReact() {
  return (
    <p>Mỗi ngày hiểu thêm một chút.</p>
  );
}
```

Dành cho Vũ và người mới bắt đầu

Biên soạn: 07/10/2026 • Có mục lục bấm được và phần sửa lỗi

Nội dung và ví dụ được viết mới; không phải bản chép lời video.

Cách học cuốn này

Bạn không cần hiểu hết cả 20 bài trong một lần.

Mục tiêu là giúp bạn **tự giải thích và sửa được code**, không chỉ chép để thấy giao diện. Các bài đi theo 20 mốc video bạn gửi. Lời giải thích và demo được viết mới bằng tiếng Việt; không phải bản dịch nguyên văn hay bản chép lời Bro Code.

Mỗi buổi học chỉ cần một vòng nhỏ

1. Đọc phần “Hiểu trước khi gõ”. Nói lại bằng một câu: bài này giải quyết việc gì?

2. Tự gõ code theo đúng tên file. Đọc cả dòng import và đoạn App.jsx.

3. Thực hiện mục “Kết quả cần thấy”. Nếu khác, mở Console trước khi sửa lung tung.

4. Đổi một giá trị, dự đoán kết quả rồi mới chạy. Làm bài tập ngắn cuối bài.

Khi gặp từ này	Hãy hiểu là
component	Một phần giao diện được viết thành hàm: thẻ cá nhân, bộ đếm, danh sách.
props	Dữ liệu component cha truyền xuống component con.
state	Dữ liệu React ghi nhớ cho component; cập nhật nó có thể làm giao diện đổi.
render / re-render	React gọi component để tính giao diện lần đầu / tính lại.
hook / handler	Hook: hàm use... của React. Handler: hàm xử lý thao tác như bấm, nhập.

QUY ƯỚC QUAN TRỌNG

Mỗi lần chỉ mở **một demo** bằng `App.jsx`. Không dán 20 component vào cùng một file. Bài 01 tạo bộ khung dùng chung; các bài sau thay component đang mở. Demo giữ dữ liệu trong bộ nhớ, nên tải lại trang sẽ về trạng thái ban đầu.

Nếu đang vướng ở to-do list, hãy học chắc bài 08, 09, 11, 13 và 14 rồi quay lại bài 15. Bạn có thể bấm số trang trong mục lục hoặc dùng thanh Bookmarks của trình đọc PDF.

Mục lục 20 bài

Bấm tên bài để tới trang PDF; bấm mốc thời gian để mở video tại đúng đoạn.

BÀI	NỘI DUNG	VIDEO	TRANG
01	React, JSX và tạo project	00:00:00	6
02	Card component	00:20:26	10
03	Thêm CSS cho component	00:32:24	12
04	Props và giá trị mặc định	00:40:40	14
05	Hiển thị có điều kiện	00:52:49	16
06	Render danh sách	01:03:04	18
07	Sự kiện click	01:29:43	20
08	Hook useState	01:42:04	22
09	Sự kiện onChange	01:58:36	25
10	Ứng dụng chọn màu	02:13:16	28
11	Updater functions	02:23:31	30
12	Cập nhật object trong state	02:30:45	33
13	Cập nhật array trong state	02:39:55	36
14	Cập nhật array chứa object	02:48:55	39
15	Ứng dụng To-do List	03:01:42	43
16	Hook useEffect	03:24:17	48
17	Đồng hồ điện tử	03:44:08	52
18	Hook useContext	04:00:08	55
19	Hook useRef	04:11:44	58
20	Đồng hồ bấm giờ	04:23:01	61

Mở video React Full Course - Bro Code trên YouTube

NÊN ĐỌC TRƯỚC

Hai trang JavaScript ngay sau mục lục giúp bạn hiểu dấu `{}`, `=>`, `...`, `map()` và `filter()`. Cuối tài liệu có phần chữa trang trắng, bảng chọn hook và bài luyện tập có đáp án.

JavaScript cần biết: phần 1

Không phải cú pháp lạ nào trong React cũng là cú pháp của React.

1. Hàm thường và arrow function

JavaScript minh họa - đọc trước, không dán vào App.jsx

```
function double(number) {
  return number * 2;
}

const doubleShort = (number) => number * 2;
const doubleLong = (number) => {
  return number * 2;
};
```

Cả ba hàm đều nhận một số và trả về số gấp đôi. Với arrow function, viết ngay một biểu thức sau `=>` thì trả về biểu thức đó. Nếu mở thân hàm bằng `{`, thường cần viết `return` khi muốn trả kết quả.

2. Object và destructuring: lấy dữ liệu ra cho gọn

Hai cách đọc cùng một object

```
const student = { name: "Vũ", age: 20 };
const name1 = student.name;

const { name, age } = student;
console.log(name, age); // Vũ, 20

const pair = [7, "xin chào"];
const [number, message] = pair;
```

`{ name, age }` lấy thuộc tính theo **tên**. `[number, message]` lấy phần tử theo **vị trí**. Bài `useState` dùng kiểu thứ hai để nhận hai thứ: giá trị hiện tại và hàm cập nhật.

3. Ba kiểu dấu ngoặc

Ký hiệu	Ví dụ	Ý nghĩa ở ví dụ
`()`	`greet("Vũ")`	Gọi hàm; cũng có thể nhóm biểu thức.
`[]`	`["Táo", "Cam"]`	Mảng; cũng dùng truy cập phần tử như arr[0].
`{}`	`{ name: "Vũ" }`	Object; trong JSX còn dùng để chèn JavaScript.

CÙNG KÝ HIỆU, KHÁC NGỮ CẢNH

`function test() { ... }` dùng `{}` làm thân hàm. `const user = { ... }` dùng `{}` tạo object. Đừng học thuộc một nghĩa duy nhất cho dấu ngoặc.

JavaScript cần biết: phần 2

Các thao tác sẽ lặp đi lặp lại từ bài 06 tới bài 15.

Chạy thử từng đoạn trong Console để xem kết quả

```
const scores = [4, 6, 8];
const doubled = scores.map((score) => score * 2);
// doubled: [8, 12, 16]

const passed = scores.filter((score) => score >= 5);
// passed: [6, 8]

const moreScores = [...scores, 10];
// moreScores: [4, 6, 8, 10]; scores không bị sửa

const car = { brand: "Toyota", year: "2024" };
const nextCar = { ...car, year: "2025" };
// Giữ brand, thay year; car vẫn là object cũ

const message = `Bạn có ${scores.length} điểm số.`;
```

Công cụ	Câu hỏi nó trả lời
`map()`	Với mỗi phần tử, tôi muốn biến nó thành gì?
`filter()`	Tôi muốn giữ những phần tử thỏa điều kiện nào?
`...` trong ví dụ	Tôi muốn lấy các phần tử / thuộc tính hiện có ra.
Dấu backtick	Tôi muốn ghép chuỗi với biến bằng ${...}.

Toán tử điều kiện thường gặp

```
const isLoggedIn = true;
const title = isLoggedIn ? "Xin chào" : "Đăng nhập";
// Điều kiện ? kết quả khi đúng : kết quả khi sai

const name = "";
const displayedName = name || "Khách";
// Chuỗi rỗng là falsy nên nhận "Khách"
```

ĐIỂM DỄ SAI

`map()` và `filter()` tạo mảng mới. Nhưng `sort()` sửa trực tiếp mảng được gọi. Nếu muốn giữ mảng gốc, hãy dùng `[...array].sort(...)`. Spread chỉ sao chép nông: object nằm trong mảng vẫn cần được sao chép riêng nếu bạn sửa nó.

TỰ LÀM TRONG 5-10 PHÚT

Từ [2, 5, 9], tạo mảng chỉ giữ số lớn hơn 4.
Gợi ý: Dùng filter; đáp án là [5, 9]. Sau đó thử dùng map để cộng 1 vào từng số.

Đọc thêm: R18, R30. Liên kết đầy đủ ở cuối tài liệu.

React, JSX và tạo project

Video 00:00:00 | Đích đến: chạy được trang React đầu tiên.

Hiểu trước khi gõ

React là **thư viện JavaScript để xây giao diện**. JSX là cách viết giao diện gần giống HTML trong file JavaScript. Vite giúp tạo project, chạy môi trường phát triển và đóng gói code. Node.js chạy các công cụ đó trên máy; giao diện React của các bài này chạy trong trình duyệt.

Phần	Vai trò trong bài học
HTML / JSX	Có tiêu đề, ô nhập, nút bấm nào?
CSS	Màu sắc, khoảng cách, bố cục ra sao?
JavaScript + React	Bấm nút thì dữ liệu và giao diện đổi thế nào?

Bước 1. Cài và kiểm tra công cụ

Cài Node.js bản LTS đang được hỗ trợ từ nodejs.org, rồi mở lại VS Code. Mở Terminal > New Terminal. Tài liệu Vite hiện nêu yêu cầu Node.js 20.19+ hoặc 22.12+; chọn bản LTS mới còn hỗ trợ và tuân theo cảnh báo phiên bản nếu template yêu cầu cao hơn. [R01]

Terminal - nhập từng dòng

```
node -v
npm -v
npm create vite@latest react-vietnamese -- --template react
cd react-vietnamese
npm install
npm run dev
```

Nếu được hỏi tải công cụ tạo project, đồng ý. Lệnh đã chọn template React dùng JavaScript. Mở đúng địa chỉ “Local” mà Terminal in ra, thường là `http://localhost:5173`. Giữ Terminal này chạy; dùng Ctrl+C để dừng.

> **NẾU BẠN ĐÃ CÓ PROJECT VITE CHẠY ĐƯỢC**
>
> Không cần tạo project mới. Mở đúng thư mục chứa `package.json`, rồi làm các bước chỉnh file ở trang sau. Nếu PowerShell chặn `npm.ps1`, có thể mở Terminal dạng Command Prompt và chạy lại lệnh npm.

Đọc thêm: R01, R02. Liên kết đầy đủ ở cuối tài liệu.

Bộ khung tối thiểu

Bước 2. Giữ đúng đường nối: index.html, main.jsx, App.jsx, component.

File	Việc của file
index.html	Chứa chỗ trống có id="root" và nạp main.jsx.
src/main.jsx	Gắn component App vào chỗ trống đó.
src/App.jsx	Chọn demo đang học để hiển thị.
src/index.css	CSS dùng chung cho toàn bộ demo.

index.html - ở thư mục project, bên ngoài src

```
<!doctype html>
<html lang="vi">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Học React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/main.jsx - dùng chung, chỉ cần thiết lập một lần

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`createRoot(...)` nhận phần tử DOM mà React sẽ quản lý. `<App />` là component đầu vào. `StrictMode` giúp phát hiện một số lỗi khi phát triển; bạn cứ giữ nguyên. Các hook vẫn phải được import trong từng file sử dụng chúng.

CHỈ GIỮ MỘT ĐIỂM IMPORT CSS NỀN

Trong tài liệu, main.jsx import index.css. Các App.jsx bên dưới không import App.css của template. Bạn có thể giữ file App.css cũ trên đĩa, nhưng bỏ dòng import nó để CSS mẫu không làm lệch bố cục.

Đọc thêm: R31, R22. Liên kết đầy đủ ở cuối tài liệu.

CSS nền dùng cho cả cuốn

Bước 3. Thay toàn bộ src/index.css bằng đoạn dưới. Các bài sau chỉ thêm CSS khi được yêu cầu.

src/index.css - CSS nền

```
* { box-sizing: border-box; }
body {
  margin: 0;
  background: #f1f5f9;
  color: #172033;
  font-family: Arial, sans-serif;
}
#root { max-width: 820px; margin: 36px auto; padding: 0 16px; }
.panel {
  background: white;
  border: 1px solid #dbe2ea;
  border-radius: 14px;
  padding: 24px;
  margin-bottom: 16px;
}
h1, h2 { margin-top: 0; }
button, input, select, textarea { font: inherit; }
button {
  padding: 9px 14px;
  border: 0;
  border-radius: 8px;
  background: #2563eb;
  color: white;
  cursor: pointer;
}
button:disabled { opacity: 0.45; cursor: not-allowed; }
input:not([type="checkbox"]):not([type="color"]),
select, textarea {
  padding: 9px;
  border: 1px solid #94a3b8;
  border-radius: 8px;
  max-width: 100%;
}
label { display: grid; gap: 6px; margin: 12px 0; }
.row { display: flex; flex-wrap: wrap; align-items: center; gap: 8px; }
li { margin: 10px 0; }
.number { font-size: 48px; margin: 16px 0; }
```

CSS này tạo nền sáng, khung trắng và nút xanh đơn giản. `.panel` là khung nội dung; `.row` xếp các phần tử cùng hàng và tự xuống dòng; `.number` là số lớn của bộ đếm.

> **CSS KHÔNG PHẢI ĐIỀU KIỆN ĐỂ REACT RENDER**
>
> Nếu chưa thêm CSS riêng của một bài, nội dung vẫn phải xuất hiện. Một trang trắng hoàn toàn thường cần kiểm tra import, JSX hoặc lỗi JavaScript trước.

Chạy component đầu tiên

Bước 4. Tạo HelloReact.jsx, rồi nối nó vào App.jsx.

src/HelloReact.jsx - tạo file và dán toàn bộ

```
export default function HelloReact() {
  const name = "Vũ";

  return (
    <main className="panel">
      <h1>Xin chào, {name}!</h1>
      <p>Mình đang học React từng bước.</p>
    </main>
  );
}
```

src/App.jsx - thay toàn bộ nội dung

```
import HelloReact from "./HelloReact.jsx";

export default function App() {
  return <HelloReact />;
}
```

Đọc code theo thứ tự

1. `HelloReact` là hàm viết hoa chữ đầu. `export default` cho phép file khác import nó.

2. `const name = "Vũ"` tạo dữ liệu. `{name}` chèn dữ liệu ấy vào JSX.

3. `return (...)` trả về giao diện. Dấu `(` nằm cùng dòng với return để tránh lỗi tự thêm dấu chấm phẩy.

4. `<HelloReact />` yêu cầu React hiển thị component. Chỉ tạo file mà chưa dùng nó trong App thì bạn chưa thấy nó.

Quy tắc JSX	Ví dụ đúng
Bọc các thẻ cùng cấp	`<> <h1>...</h1> <p>...</p> </>`
Đóng thẻ	`<input />`, `<img />`, `<p>Chào</p>`
Dùng className	`<div className="panel">...</div>`
Chèn JavaScript bằng {}	`<p>{2 + 3}</p>` hiển thị 5.

TỰ LÀM TRONG 5-10 PHÚT

Đổi tên thành tên bạn, rồi thêm một dòng giới thiệu ngành học.
Gợi ý: Thêm một thẻ p bên trong main. Kết quả phải vẫn có tiêu đề và cả hai dòng mô tả.

Đọc thêm: R02, R03, R04, R05. Liên kết đầy đủ ở cuối tài liệu.

Card component

Video 00:20:26 | Component là một phần giao diện có thể dùng lại.

Hãy tưởng tượng trang web có nhiều thẻ sinh viên. Một thẻ gồm hình đại diện, tên và mô tả. Bạn viết hình dạng thẻ một lần trong `Card`, rồi đặt `<Card />` ở nơi cần dùng. Ở bài này dữ liệu còn cố định; bài 04 sẽ giúp mỗi thẻ nhận dữ liệu khác nhau.

1. Tạo `src/Card.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import Card from "./Card.jsx";

export default function App() {
  return (
    <>
      <Card />
      <Card />
    </>
  );
}
```

Kết quả cần thấy

Hai thẻ giống nhau, cùng tên Trương Hoàng Vũ. Điều này bình thường vì nội dung Card đang viết cố định. Hai lần dùng component là hai vị trí hiển thị, không phải bạn vô tình tạo hai file.

Cú pháp	Ý nghĩa
`function Card()`	Định nghĩa component.
`<Card />`	Dùng component trong giao diện.
`<article>`	Thẻ HTML có sẵn; tên viết thường.
`import Card from "./Card.jsx"`	Lấy default export từ file ở cùng thư mục.

MỘT FILE, MỘT DEFAULT EXPORT

Bạn có thể đặt nhiều hàm trong một file, nhưng chỉ có một export default. Trong cuốn này, tên component thường trùng tên file để dễ tìm. Đừng viết định nghĩa Card bên trong App.

Đọc thêm: R02, R04. Liên kết đầy đủ ở cuối tài liệu.

Tự làm thẻ cá nhân

Không cần tải ảnh ngoài để chạy được ví dụ.

src/Card.jsx - tạo file và dán toàn bộ

```
export default function Card() {
  return (
    <article className="panel">
      <div className="avatar">VU</div>
      <h2>Trương Hoàng Vũ</h2>
      <p>Sinh viên Công nghệ thông tin</p>
      <p>Mục tiêu: tự làm giao diện React.</p>
    </article>
  );
}
```

src/index.css - thêm ở cuối, giữ CSS nền ở bài 01

```
.avatar {
  width: 72px; height: 72px; border-radius: 50%;
  display: grid; place-items: center;
  background: #dbeafe; color: #1d4ed8; font-weight: bold;
}
```

Khối `.avatar` dùng hai chữ VU thay hình đại diện. `border-radius: 50%` làm nó tròn; `display: grid` và `place-items: center` đặt chữ ở giữa. Đây chỉ là CSS thông thường.

Nếu muốn thay bằng ảnh của bạn

Tùy chọn - đặt ảnh avatar.jpg vào public, thay div avatar

```
<img src="/avatar.jpg" alt="Ảnh đại diện của Vũ"
  width="72" height="72" />
```

File nằm ở `public/avatar.jpg` thì đường dẫn trong ví dụ là `/avatar.jpg`, không phải `/public/avatar.jpg`. Cách này dành cho demo ở gốc website; khi triển khai vào đường dẫn con cần xem cấu hình base của Vite.

TỰ LÀM TRONG 5-10 PHÚT

Hiển thị ba thẻ và đổi một dòng giới thiệu trong Card.
Gợi ý: Thêm một <Card /> trong App. Đổi Card một lần thì cả ba thẻ đều đổi nội dung.

Đọc thêm: R02, R34. Liên kết đầy đủ ở cuối tài liệu.

Thêm CSS cho component

Video 00:32:24 | React vẫn dùng CSS, chỉ có cách nối CSS với JSX khác một chút.

Cách viết	Khi nào dễ dùng	Điểm cần nhớ
CSS ngoài	Màu, font, khung, khoảng cách dùng chung.	className="lesson-title" khớp .lesson-title trong CSS.
CSS Module	CSS riêng cho một component.	Tên file có .module.css; import object styles.
Inline style	Một giá trị style phụ thuộc dữ liệu.	Truyền object; dùng backgroundColor thay background-color.

1. Tạo `src/StyleDemo.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import StyleDemo from "./StyleDemo.jsx";

export default function App() {
  return <StyleDemo />;
}
```

src/StyleDemo.module.css - tạo file mới

```
.button {
  background: #0f766e;
  border-radius: 999px;
  padding: 12px 20px;
}
```

src/index.css - thêm ở cuối, giữ CSS nền ở bài 01

```
.lesson-title { color: #1d4ed8; }
```

DẤU {{ }} KHÔNG PHẢI "CÚ PHÁP HAI NGOẶC"

Trong `style={{ color: "red" }}`, cặp ngoài mở vùng JavaScript của JSX; cặp trong tạo object JavaScript. Có thể tách object ra biến rồi viết `style={noteStyle}` như ở trang sau.

Đọc thêm: R05, R06. Liên kết đầy đủ ở cuối tài liệu.

Demo ba cách thêm CSS

Tạo StyleDemo.jsx sau khi đã tạo file CSS Module ở trang trước.

src/StyleDemo.jsx - tạo file và dán toàn bộ

```
import styles from "./StyleDemo.module.css";

export default function StyleDemo() {
  const noteStyle = {
    backgroundColor: "#e0f2fe",
    padding: "12px",
    borderRadius: "8px",
  };

  return (
    <main className="panel">
      <h1 className="lesson-title">Ba cách viết CSS</h1>
      <p style={noteStyle}>Đây là inline style.</p>
      <button className={styles.button}>
        Đây là CSS Module
      </button>
    </main>
  );
}
```

Kết quả cần thấy

• Tiêu đề xanh dương do `.lesson-title` trong index.css.

• Khung chú thích xanh nhạt do object noteStyle.

• Nút xanh ngọc, bo tròn mạnh do styles.button trong CSS Module.

`styles.button` là tên class do Vite xử lý từ file module. Đừng viết `className="styles.button"`: cách đó chỉ truyền một chuỗi chữ nguyên xi, không đọc thuộc tính của object styles.

TỰ LÀM TRONG 5-10 PHÚT

Đổi backgroundColor của noteStyle thành "#fef3c7".
Gợi ý: Khung ghi chú chuyển vàng nhạt; các chỗ khác giữ màu. Sau đó đổi padding từ "12px" thành "24px" và quan sát.

LỖI THƯỜNG GẶP

Không viết `style="color: red"` như HTML thuần. React cần object, ví dụ `style={{ color: "red" }}`. CSS ngoài vẫn dùng dấu gạch nối như background-color; chỉ object JavaScript mới dùng backgroundColor.

Đọc thêm: R05, R06. Liên kết đầy đủ ở cuối tài liệu.

Props và giá trị mặc định

Video 00:40:40 | Props là dữ liệu truyền từ cha xuống con.

Ở bài thẻ cá nhân, mọi thẻ có cùng dữ liệu. Với props, component giống một mẫu đơn trống: cha cung cấp tên, tuổi và trạng thái sinh viên; con dùng dữ liệu đó để hiển thị. App là cha, Student là con.

src/Student.jsx - tạo file và dán toàn bộ

```
export default function Student({
  name = "Khách",
  age = 18,
  isStudent = false,
}) {
  return (
    <article className="panel">
      <h2>{name}</h2>
      <p>Tuổi: {age}</p>
      <p>Sinh viên: {isStudent ? "Có" : "Không"}</p>
    </article>
  );
}
```

Đọc phần nhận props

React truyền một object props vào component. Viết `function Student({ name, age })` là lấy sẵn hai thuộc tính từ object đó. Nó tương đương nhận `props` rồi đọc `props.name`, `props.age`.

Bạn viết ở App	Student nhận được
`name="Vũ"`	Chuỗi "Vũ".
`age={20}`	Số 20; khác với age="20" là chuỗi.
`isStudent={false}`	Boolean false; khác với chuỗi "false" vốn là truthy.
`<Student />`	Dùng các giá trị mặc định đã khai báo.

PROPS LÀ DỮ LIỆU ĐỂ ĐỌC

Không tự gán lại props để sửa dữ liệu của cha. Khi cần tương tác và thay đổi giao diện, bạn sẽ dùng state. Luồng cơ bản là cha truyền dữ liệu xuống con.

Đọc thêm: R07. Liên kết đầy đủ ở cuối tài liệu.

Props trên React 19

Giá trị mặc định đặt ngay trong tham số của function component.

Bước làm: tạo Student.jsx theo trang trước; thay App.jsx bằng đoạn dưới; giữ CSS nền. Chạy trang và đối chiếu ba thẻ.

src/App.jsx - thay toàn bộ nội dung

```
import Student from "./Student.jsx";

export default function App() {
  return (
    <>
      <Student name="Vũ" age={20} isStudent={true} />
      <Student name="Lan" age={21} isStudent={false} />
      <Student />
    </>
  );
}
```

Thẻ	Kết quả
Thẻ 1	Vũ / Tuổi: 20 / Sinh viên: Có
Thẻ 2	Lan / Tuổi: 21 / Sinh viên: Không
Thẻ 3	Khách / Tuổi: 18 / Sinh viên: Không

Khác biệt cần biết so với video cũ

React 19 đã **loại bỏ hỗ trợ defaultProps cho function component** và không còn chạy kiểm tra propTypes của component. Bởi vậy ví dụ trong cuốn này không import package prop-types, không dùng Student.defaultProps. Class component vẫn hỗ trợ defaultProps. [R08]

Cách hiện tại trong JavaScript

```
function Student({ name = "Khách" }) {
  return <h2>{name}</h2>;
}
```

Giá trị mặc định áp dụng khi prop bị bỏ qua hoặc là `undefined`. Nếu cha truyền `name={null}` hoặc `name=""`, giá trị đó không tự đổi thành "Khách". Nếu muốn kiểm tra kiểu chặt chẽ trong project lớn, bạn có thể học TypeScript sau.

TỰ LÀM TRONG 5-10 PHÚT

Thêm prop major với mặc định "Chưa cập nhật" rồi hiển thị ngành học.
Gợi ý: Thêm major vào phần tham số và thêm {major} . Chỉ truyền major="CNTT" cho thẻ đầu để thấy sự khác nhau.

Đọc thêm: R07, R08. Liên kết đầy đủ ở cuối tài liệu.

Hiển thị có điều kiện

Video 00:52:49 | Cùng một component, nhưng dữ liệu khác thì giao diện khác.

Conditional rendering nghĩa là **hiển thị theo điều kiện**. Ví dụ: chưa đăng nhập thì hiện lời nhắc; đã đăng nhập thì hiện tên. Đây là cách tổ chức giao diện bằng if, toán tử ba ngôi hoặc && của JavaScript.

src/UserGreeting.jsx - tạo file và dán toàn bộ

```
export default function UserGreeting({
  isLoggedIn = false,
  username = "Khách",
}) {
  if (!isLoggedIn) {
    return (
      <p className="panel">Bạn hãy đăng nhập nhé.</p>
    );
  }

  return (
    <h2 className="panel">Chào mừng, {username}!</h2>
  );
}
```

src/App.jsx - thay toàn bộ nội dung

```
import UserGreeting from "./UserGreeting.jsx";

export default function App() {
  return <UserGreeting isLoggedIn={true} username="Vũ" />;
}
```

Tutorial từng bước

1. Tạo UserGreeting.jsx và thay App.jsx theo hai đoạn trên.

2. Chạy với isLoggedIn={true}: phải thấy “Chào mừng, Vũ!”.

3. Sửa thành isLoggedIn={false}: phải thấy “Bạn hãy đăng nhập nhé.”

4. Đọc dấu !: !isLoggedIn nghĩa là “không đăng nhập”. Khi if đúng, return đầu kết thúc hàm; phần return sau không chạy.

> **ĐÂY LÀ DEMO GIAO DIỆN**
>
> Prop isLoggedIn trong ví dụ do bạn tự đặt. Nó không phải cơ chế xác thực tài khoản thật. Ứng dụng có backend cần kiểm tra quyền ở phía server.

Đọc thêm: R09. Liên kết đầy đủ ở cuối tài liệu.

Chọn if, ? : hay &&

Ba cách giải cùng một loại bài toán, nhưng mục đích diễn đạt khác nhau.

Cách	Dùng khi	Ví dụ trong JSX
if + return	Muốn tách rõ hai nhánh hoặc nhánh dài.	Như UserGreeting ở trang trước.
`? :`	Có hai lựa chọn ngắn.	`{ok ? "Đã xong" : "Chưa xong"}`
`&&`	Đúng thì hiện; sai thì không hiện phần đó.	`{isAdmin && <button>Quản lý</button>}`

Đoạn minh họa - viết trong phần return của component

```
<p>{isLoggedIn ? "Đã đăng nhập" : "Chưa đăng nhập"}</p>

{messages.length > 0 && <p>Bạn có tin nhắn mới.</p>}
```

Hai dòng minh họa trên giả sử bạn đã có biến isLoggedIn và mảng messages. Chúng không phải file chạy độc lập. Trong code hoàn chỉnh của bài, bạn chỉ cần UserGreeting.jsx và App.jsx.

Vì sao không nên viết messages.length && ...?

Nếu mảng rỗng, messages.length bằng 0; biểu thức trả 0, và React có thể hiển thị số 0. Viết `messages.length > 0` sẽ tạo boolean rõ ràng.

Cách chủ động không hiển thị gì

Một component phụ - chỉ minh họa cú pháp

```
function Notice({ show }) {
  if (!show) return null;
  return <p>Có thông báo mới.</p>;
}
```

TỰ LÀM TRONG 5-10 PHÚT

Thêm prop isAdmin={true} vào UserGreeting và chỉ hiện chữ “Quản trị viên” khi đúng. **Gợi ý:** Nhận isAdmin = false ở tham số; thêm {isAdmin && Quản trị viên } trong một thẻ bọc cùng lời chào.

ĐỪNG NHẦM CHUỖI VỚI BOOLEAN

`isLoggedIn="false"` vẫn truyền một chuỗi không rỗng, nên điều kiện được xem là đúng. Khi cần false thật, dùng `{false}`.

Đọc thêm: R09. Liên kết đầy đủ ở cuối tài liệu.

Render danh sách

Video 01:03:04 | Biến dữ liệu trong mảng thành các phần tử giao diện.

Hiểu trước khi gõ

Thay vì tự viết ba thẻ li, bạn giữ dữ liệu trong một mảng và dùng `map()` tạo một thẻ cho mỗi phần tử. Hôm sau thêm dữ liệu vào mảng thì số dòng giao diện cũng thay đổi theo.

Bước	Dữ liệu đi qua bước đó
Mảng gốc	Táo 52, Chuối 89, Cam 47.
filter: dưới 60	Còn Táo 52 và Cam 47.
sort: tăng dần	Cam 47 đứng trước Táo 52.
map: tạo JSX	Mỗi object thành một dòng có tên và số kcal.

Các con số là dữ liệu minh họa cho bài lập trình. Mục tiêu ở đây là hiểu xử lý mảng, không phải xây bảng dinh dưỡng.

1. Tạo `src/FruitList.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import FruitList from "./FruitList.jsx";

export default function App() {
  return <FruitList />;
}
```

KEY GIÚP REACT NHẬN DIỆN TỪNG DÒNG

Dùng id ổn định có trong dữ liệu, duy nhất giữa các phần tử cùng danh sách. Khi có thêm, xóa hay đổi vị trí, đừng lấy index làm danh tính. Không tạo Math.random() hoặc randomUUID() ngay trong key mỗi lần render.

Đọc thêm: R10. Liên kết đầy đủ ở cuối tài liệu.

Demo lọc và hiển thị danh sách

Kết quả: Cam 47 đứng trước Táo 52; Chuối không xuất hiện.

src/FruitList.jsx - tạo file và dán toàn bộ

```
const fruits = [
  { id: 1, name: "Táo", calories: 52 },
  { id: 2, name: "Chuối", calories: 89 },
  { id: 3, name: "Cam", calories: 47 },
];

export default function FruitList() {
  const lightFruits = fruits.filter(
    (fruit) => fruit.calories < 60
  );
  const sortedFruits = [...lightFruits].sort(
    (a, b) => a.calories - b.calories
  );

  return (
    <main className="panel">
      <h1>Trái cây dưới 60 kcal / 100 g</h1>
      <ul>
        {sortedFruits.map((fruit) => (
          <li key={fruit.id}>
            {fruit.name}: {fruit.calories} kcal
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Hai cách viết map đúng

So sánh cú pháp - giữ một trong hai cách

```
items.map((item) => <li key={item.id}>{item.name}</li>)

items.map((item) => {
  return <li key={item.id}>{item.name}</li>;
})
```

Mở `{}` sau dấu mũi tên mà quên `return` là lỗi hay làm danh sách trống. Mảng đã có dữ liệu nhưng callback không trả JSX. Biến items trong đoạn so sánh chỉ là tên minh họa; demo hoàn chỉnh dùng sortedFruits.

TỰ LÀM TRONG 5-10 PHÚT

Hiển thị tất cả trái cây, sắp xếp calories từ cao xuống thấp.
Gợi ý: Lấy [...fruits] thay vì [...lightFruits]; comparator dùng b.calories - a.calories. Kết quả: Chuối, Táo, Cam.

Đọc thêm: R10, R18. Liên kết đầy đủ ở cuối tài liệu.

Sự kiện click

Video 01:29:43 | Event handler là hàm React sẽ gọi khi người dùng thao tác.

Hàm xử lý sự kiện giống một chỉ dẫn để sẵn: “khi bấm thì làm việc này”. Bạn truyền hàm cho `onClick`; không gọi hàm ngay trong quá trình tạo giao diện.

1. Tạo `src/ClickDemo.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import ClickDemo from "./ClickDemo.jsx";

export default function App() {
  return <ClickDemo />;
}
```

Cách viết	Khi nào hàm được gọi?
`onClick={sayHello}`	Khi người dùng bấm. Đây là cách truyền hàm.
`onClick={() => greet("Lan")}`	Khi bấm, hàm bọc mới gọi greet với tham số Lan.
`onClick={sayHello()}`	Gọi ngay khi render; thường sai với handler kiểu này.

Kết quả cần thấy

1. Nút “Chào mình” mở hộp thoại “Chào Vũ!”.

2. Nút “Chào Lan” mở hộp thoại “Chào Lan!”.

3. Bấm F12, mở Console rồi bấm “Xem Console”: tên nút được in ra.

NHẤN NÚT CHƯA CHẮC LÀM GIAO DIỆN ĐỔI

Handler có thể mở hộp thoại, ghi log, gửi dữ liệu hoặc cập nhật state. Chỉ tạo một biến thường và tăng biến đó chưa phải cách để React nhớ và hiển thị số mới; bài 08 sẽ giải quyết.

Đọc thêm: R11. Liên kết đầy đủ ở cuối tài liệu.

Demo click hoàn chỉnh

alert dùng cho bài học nhỏ; không cần thư viện giao diện.

src/ClickDemo.jsx - tạo file và dán toàn bộ

```
export default function ClickDemo() {
  function sayHello() {
    alert("Chào Vũ!");
  }

  function greet(name) {
    alert(`Chào ${name}!`);
  }

  return (
    <main className="panel">
      <h1>Sự kiện click</h1>
      <div className="row">
        <button onClick={sayHello}>Chào mình</button>
        <button onClick={() => greet("Lan")}>
          Chào Lan
        </button>
        <button onClick={(event) => {
          console.log(event.currentTarget.textContent);
        }}>
          Xem Console
        </button>
      </div>
    </main>
  );
}
```

event là gì?

React truyền một object mô tả sự kiện cho handler. Tên tham số có thể là `event` hoặc `e`. Trong ví dụ, `event.currentTarget` là nút mà handler đang gắn vào. `textContent` là nội dung chữ của nút.

`event.target` là phần tử khởi phát sự kiện. Nếu nút chứa một thẻ con và bạn bấm vào thẻ con, target có thể là thẻ đó; currentTarget vẫn là nút. Với onChange của input ở bài sau, target.value cho bạn giá trị ô nhập.

TỰ LÀM TRONG 5-10 PHÚT

Tạo nút “Chào React” gọi greet("React").
Gợi ý: Dùng onClick={() => greet("React")}. Nếu hộp thoại bật ngay khi tải trang thì bạn đã gọi hàm quá sớm.

Đọc thêm: R11. Liên kết đầy đủ ở cuối tài liệu.

Hook useState

Video 01:42:04 | State là phần dữ liệu React nhớ cho mỗi lần component xuất hiện.

Vấn đề với biến thường

Đoạn sai về cách quản lý dữ liệu giao diện - chỉ để so sánh

```
let count = 0;
function increase() {
  count = count + 1;
}
```

Đoạn trên đổi một biến JavaScript, nhưng không yêu cầu React tính lại giao diện. Nếu biến nằm trong thân component, nó còn được tạo lại khi component chạy. Khi dữ liệu vừa cần được nhớ vừa cần hiện lên màn hình, dùng state.

Một dòng quan trọng

```
const [count, setCount] = useState(0);
```

Thành phần	Nghĩa dễ nhớ
`useState(0)`	Khởi tạo bộ đếm từ 0 khi component được tạo.
`count`	Giá trị dành cho lần render hiện tại.
`setCount`	Hàm yêu cầu React lưu giá trị mới cho lần render tiếp theo.
Dấu []	Tách hai phần tử của mảng kết quả do useState trả về.

Một lần bấm nút diễn ra thế nào?

1. Ban đầu count = 0. JSX tạo dòng số 0 và một nút.

2. Bạn bấm nút. Handler gọi setCount(1).

3. React xử lý cập nhật, gọi lại component với count = 1.

4. React cập nhật phần giao diện cần đổi; bạn thấy số 1.

CONST KHÔNG LÀM STATE “BẤT BIẾN MÃI MÃI”

Bạn không gán lại biến count của lần chạy hiện tại. React giữ state bên ngoài lần gọi hàm và cung cấp giá trị mới ở lần render sau. Vì thế `const [count, setCount]` vẫn phù hợp.

Đọc thêm: R12, R13, R14, R32. Liên kết đầy đủ ở cuối tài liệu.

Bộ đếm đầu tiên

Tạo Counter.jsx; giữ CSS nền đã có .number.

src/Counter.jsx - tạo file và dán toàn bộ

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  function increase() {
    setCount(count + 1);
  }

  function decrease() {
    setCount(count - 1);
  }

  return (
    <main className="panel">
      <h1>Bộ đếm</h1>
      <p className="number">{count}</p>
      <div className="row">
        <button onClick={decrease}>Giảm</button>
        <button onClick={() => setCount(0)}>Đặt lại</button>
        <button onClick={increase}>Tăng</button>
      </div>
    </main>
  );
}
```

src/App.jsx - thay toàn bộ nội dung

```
import Counter from "./Counter.jsx";

export default function App() {
  return <Counter />;
}
```

Lưu cả hai file. Chạy `npm run dev` nếu server chưa chạy. Mỗi lần bấm Tăng, số tăng 1; Giảm làm giảm 1; Đặt lại đưa số về 0. Ở demo cơ bản này, số âm được cho phép.

Đọc thêm: R13. Liên kết đầy đủ ở cuối tài liệu.

Đọc state cho đúng

Đây là nền tảng cần hiểu trước khi sang object, array và các app nhỏ.

1. State là ảnh chụp của một lần render

Đổi tạm hàm increase để quan sát trong Console

```
function increase() {
  setCount(count + 1);
  console.log(count);
}
```

Nếu màn hình đang là 0, lần bấm đầu có thể in 0 trong handler, dù giao diện sau đó hiện 1. Setter không sửa biến count trong hàm đang chạy. Tránh học thuộc “setter chạy chậm”; ý chính là **giá trị thuộc lần render nào**. Bài 11 giải thích cách cập nhật liên tiếp.

2. Mỗi component có state riêng

Thay phần return trong App.jsx để thử hai bộ đếm

```
return (
  <>
    <Counter />
    <Counter />
  </>
);
```

Bấm bộ đếm trên không tăng bộ đếm dưới. React gắn state với từng vị trí component trong cây giao diện. Khi component bị gỡ rồi tạo lại, state của instance mới được khởi tạo lại.

3. Quy tắc dùng các hook trong cuốn này

- Import hook trước khi dùng: import { useState } from "react".
- Gọi hook ở cấp cao nhất trong component hoặc custom hook, trước các return có điều kiện.
- Không đặt useState bên trong if, for, handler hoặc một hàm JavaScript tùy ý.

TỰ LÀM TRONG 5-10 PHÚT

Không cho bộ đếm giảm dưới 0.
Gợi ý: Trong decrease, dùng setCount(Math.max(0, count - 1)). Thử giảm ở 0: màn hình vẫn là 0.

DỮ LIỆU KHÔNG TỰ LƯU VĨNH VIỄN

F5 tải lại ứng dụng sẽ khởi tạo state lại. useState không tự ghi vào file, localStorage hay database. Đây là hành vi bình thường của các demo trong cuốn này.

Đọc thêm: R13, R14. Liên kết đầy đủ ở cuối tài liệu.

Sự kiện onChange

Video 01:58:36 | Đọc dữ liệu người dùng đang nhập bằng onChange.

Controlled input là ô nhập mà giá trị hiển thị được điều khiển bằng state. Cặp thường đi cùng nhau là `value={name}` và `onChange={...}`. Bạn nhập chữ; handler lấy chữ mới; setter cập nhật state; React render lại ô nhập và phần xem trước.

Loại phần tử	Giá trị điều khiển	Đọc từ sự kiện
Text, number, select	value	event.target.value
Checkbox	checked	event.target.checked

1. Tạo `src/ProfileForm.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import ProfileForm from "./ProfileForm.jsx";

export default function App() {
  return <ProfileForm />;
}
```

Demo này làm được gì?

• Nhập tên thì câu chào đổi ngay.

• Đổi số buổi thì số tiền tạm tính đổi theo.

• Chọn React hoặc Java; tick ô đồng ý và xem trạng thái.

Ô NUMBER VẪN TRẢ VỀ CHUỖI

Trong demo, quantity được giữ dạng chuỗi để bạn có thể xóa rỗng ô nhập. Chỉ khi tính tiền mới gọi Number(quantity). Chuỗi rỗng chuyển thành 0. Đây là phần xem trước; nếu gửi dữ liệu thật cần kiểm tra số nguyên dương trước khi gửi.

Code ở hai trang sau là **hai phần liên tiếp của cùng một file**. Dán phần 2 ngay sau phần 1; không tạo hai ProfileForm.jsx và không bỏ các dấu đóng ở cuối phần 2.

Đọc thêm: R15. Liên kết đầy đủ ở cuối tài liệu.

Form: state và các ô nhập

src/ProfileForm.jsx - phần 1/2. File sẽ hoàn chỉnh sau khi nối phần 2.

src/ProfileForm.jsx - phần 1/2

```
import { useState } from "react";

export default function ProfileForm() {
  const [name, setName] = useState("");
  const [quantity, setQuantity] = useState("1");
  const [course, setCourse] = useState("react");
  const [agreed, setAgreed] = useState(false);

  function handleQuantity(event) {
    setQuantity(event.target.value);
  }

  const total = Number(quantity) * 50000;

  return (
    <main className="panel">
      <h1>Phiếu đăng ký học</h1>
      <label>
        Họ tên
        <input value={name} onChange={(event) => {
          setName(event.target.value);
        }} />
      </label>
      <label>
        Số buổi
        <input type="number" min="1" value={quantity}
          onChange={handleQuantity} />
      </label>
```

Giải thích các quyết định trong code

Tên bắt đầu là chuỗi rỗng để input luôn có một giá trị chuỗi. Số buổi bắt đầu là "1". Môn học dùng mã "react"; dòng option ở phần 2 đổi mã đó thành nhãn dễ đọc. agreed là boolean nên khởi tạo bằng false.

total được tính trực tiếp từ quantity. Không cần một state total riêng, vì nó có thể suy ra từ dữ liệu đã có. Cách này tránh trường hợp số buổi đổi mà bạn quên cập nhật tổng tiền.

Đọc thêm: R15, R21. Liên kết đầy đủ ở cuối tài liệu.

Form: lựa chọn và phần xem trước

Nối đoạn dưới vào ngay sau phần 1, rồi lưu file.

src/ProfileForm.jsx - phần 2/2

```
      <label>
        Môn học
        <select value={course} onChange={(event) => {
          setCourse(event.target.value);
        }}>
          <option value="react">React</option>
          <option value="java">Java</option>
        </select>
      </label>
      <label className="row">
        <input type="checkbox" checked={agreed}
          onChange={(event) => {
            setAgreed(event.target.checked);
          }} />
        Tôi đồng ý đăng ký
      </label>
      <p>Chào {name || "bạn"}. Môn đã chọn: {course}</p>
      <p>Tạm tính: {total.toLocaleString("vi-VN")} đ</p>
      <p>Đồng ý: {agreed ? "Có" : "Chưa"}</p>
    </main>
  );
}
```

Kết quả cần thấy

Nhập tên Vũ, chọn 3 buổi, chọn Java, tick đồng ý: câu chào chứa Vũ, mã môn là java, tiền tạm tính 150.000 đ, đồng ý là Có. Dấu phân nhóm số do toLocaleString("vi-VN") xử lý.

TỰ LÀM TRONG 5-10 PHÚT

Thêm một ô textarea để nhập ghi chú và hiển thị ghi chú ở dưới.
Gợi ý: Tạo const [note, setNote] = useState(""); rồi dùng <textarea value={note} onChange={(e) => setNote(e.target.value)} /> và {note} .

VÌ SAO KHÔNG GÕ ĐƯỢC?

Nếu có value={name} nhưng không có handler cập nhật name, React luôn giữ ô ở giá trị cũ. Với checkbox, dùng checked thay vì value để điều khiển dấu tick. Thuộc tính min của input không thay thế kiểm tra dữ liệu trong JavaScript.

Đọc thêm: R15. Liên kết đầy đủ ở cuối tài liệu.

Ứng dụng chọn màu

Video 02:13:16 | Ghép useState, onChange và inline style thành một ứng dụng nhỏ.

Từ yêu cầu đến cách lưu dữ liệu

Bạn cần một ô chọn màu, một vùng xem trước và một dòng mã màu. Cả ba cùng phụ thuộc vào **một state color**. Khi ô chọn màu thay đổi, cập nhật color; vùng xem trước và mã màu sẽ tự được tính lại từ cùng dữ liệu.

1. Tạo `src/ColorPicker.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import ColorPicker from "./ColorPicker.jsx";

export default function App() {
  return <ColorPicker />;
}
```

src/index.css - thêm ở cuối, giữ CSS nền ở bài 01

```
.color-preview {
  width: 100%; height: 180px;
  border: 1px solid #94a3b8; border-radius: 12px;
}
input[type="color"] { width: 80px; height: 44px; cursor: pointer; }
```

Kết quả cần thấy

Ban đầu vùng xem trước màu xanh #60a5fa. Bấm ô chọn màu, chọn màu khác: nền vùng xem trước và mã màu bên dưới thay đổi cùng nhau. Kiểu hộp thoại chọn màu có thể khác giữa các trình duyệt.

TỰ LÀM TRONG 5-10 PHÚT

Thêm nút “Màu mặc định”.
Gợi ý: Trong JSX, thêm button với onClick={() => setColor("#60a5fa")}. Không cần thêm state thứ hai.

Đọc thêm: R05, R13, R15. Liên kết đầy đủ ở cuối tài liệu.

ColorPicker hoàn chỉnh

Tạo src/ColorPicker.jsx theo đúng chữ hoa; App import đúng tên này.

src/ColorPicker.jsx - tạo file và dán toàn bộ

```
import { useState } from "react";

export default function ColorPicker() {
  const [color, setColor] = useState("#60a5fa");

  return (
    <main className="panel">
      <h1>Chọn màu</h1>
      <div className="color-preview"
        style={{ backgroundColor: color }} />
      <p>Mã màu: <strong>{color}</strong></p>
      <label>
        Bấm vào ô bên dưới để đổi màu
        <input type="color" value={color}
          onChange={(event) => {
            setColor(event.target.value);
          }} />
      </label>
    </main>
  );
}
```

Dòng code	Vai trò
`useState("#60a5fa")`	Giữ mã màu đang chọn.
`value={color}`	Ô chọn màu đọc cùng state.
`setColor(event.target.value)`	Đưa mã màu mới vào state.
`style={{ backgroundColor: color }}`	Đổi nền vùng xem trước theo state.

MỘT NGUỒN DỮ LIỆU LÀ ĐỦ

Không tạo previewColor và selectedColor riêng nếu chúng luôn phải giống nhau. Việc lưu cùng một thông tin hai lần làm bạn dễ quên đồng bộ một bên.

Nếu màn hình trắng: kiểm tra export default, import ColorPicker trong App, và return có JSX thật. CSS thiếu chỉ làm giao diện chưa đẹp; nó không giải thích được lỗi import hoặc ReferenceError.

Updater functions

Video 02:23:31 | Khi giá trị mới phụ thuộc giá trị trước đó, updater giúp xử lý đúng hàng đợi cập nhật.

Hai nút trông gần giống nhưng kết quả khác nhau

Giả sử count đang bằng 0. Gọi `setCount(count + 1)` ba lần trong cùng handler là gửi ba yêu cầu "đặt thành 1". Gọi `setCount(previous => previous + 1)` ba lần là gửi ba phép biến đổi, mỗi phép nhận kết quả đang chờ của phép trước.

Cách	Lượt 1	Lượt 2	Lượt 3	Kết quả
Truyền giá trị	đặt 1	đặt 1	đặt 1	1
Truyền updater	0 + 1	1 + 1	2 + 1	3

1. Tạo `src/UpdaterDemo.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import UpdaterDemo from "./UpdaterDemo.jsx";

export default function App() {
  return <UpdaterDemo />;
}
```

PREVIOUS KHÔNG PHẢI TỪ KHÓA

Đó chỉ là tên tham số do bạn đặt. Có thể viết prev, current hoặc c. React truyền giá trị state đang chờ xử lý vào tham số ấy; bạn trả lại giá trị mới.

Đọc thêm: R16, R14. Liên kết đầy đủ ở cuối tài liệu.

Thử hai cách cập nhật

Mỗi lần thử, bấm Đặt lại về 0 để so sánh dễ hơn.

src/UpdaterDemo.jsx - tạo file và dán toàn bộ

```
import { useState } from "react";

export default function UpdaterDemo() {
  const [count, setCount] = useState(0);

  function addWithValue() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }

  function addWithUpdater() {
    setCount((previous) => previous + 1);
    setCount((previous) => previous + 1);
    setCount((previous) => previous + 1);
  }

  return (
    <main className="panel">
      <h1>So sánh hai cách cập nhật</h1>
      <p className="number">{count}</p>
      <div className="row">
        <button onClick={addWithValue}>Truyền giá trị</button>
        <button onClick={addWithUpdater}>Truyền hàm</button>
        <button onClick={() => setCount(0)}>Đặt lại</button>
      </div>
    </main>
  );
}
```

Bấm “Truyền giá trị” từ 0 thì thấy 1. Đặt lại, bấm “Truyền hàm” thì thấy 3. Bấm nút truyền hàm lần nữa thì thấy 6. Đây là khác biệt do cách tính state tiếp theo, không phải do nút thứ hai chạy nhanh hơn.

Đọc thêm: R16. Liên kết đầy đủ ở cuối tài liệu.

Dùng updater đúng chỗ

Đây là cách viết xuất hiện nhiều trong các bài object và array tiếp theo.

Bạn muốn	Cách viết dễ đọc
Thay tên bằng dữ liệu vừa nhập	`setName(event.target.value)`
Đặt lại số về 0	`setCount(0)`
Tăng từ số hiện có	`setCount(prev => prev + 1)`
Đảo boolean	`setOpen(prev => !prev)`
Thêm vào mảng hiện có	`setItems(prev => [...prev, newItem])`

Updater phải thuần

Hàm thuần ở đây có nghĩa: nhận state trước và trả state sau; không sửa trực tiếp state trước, không gửi yêu cầu mạng, không tạo tác dụng phụ. React có thể gọi updater thêm trong chế độ phát triển để kiểm tra lỗi. Vì vậy đừng đặt alert, lưu database hoặc tạo ID ngẫu nhiên bên trong updater.

Ví dụ: tạo dữ liệu mới trong handler, rồi đưa vào updater

```
function addItem() {
  const item = { id: crypto.randomUUID(), text: "Học bài" };
  setItems((previous) => [...previous, item]);
}
```

Đoạn minh họa giả sử bạn đã có state items. ID được tạo một lần lúc thêm phần tử, không tạo mới mỗi lần render. randomUUID() dùng được trong môi trường an toàn như localhost hoặc HTTPS trên trình duyệt hỗ trợ.

Có bắt buộc mọi setter đều dùng updater không?

Không. Nếu giá trị mới đã có sẵn, như chuỗi đọc từ ô nhập hoặc số 0 để reset, truyền trực tiếp rất rõ ràng. Nếu bạn tính từ state trước, updater thường giúp diễn đạt đúng ý hơn.

TỰ LÀM TRONG 5-10 PHÚT

Thêm nút “Tăng 5” dùng một updater.
Gợi ý: setCount(previous => previous + 5). Không cần gọi setter năm lần.

Đọc thêm: R13, R16, R33. Liên kết đầy đủ ở cuối tài liệu.

Cập nhật object trong state

Video 02:30:45 | Một state có thể chứa nhiều thuộc tính liên quan nhau.

Thông tin một chiếc xe có hãng, mẫu và năm sản xuất. Object giúp nhóm chúng thành một giá trị car. Khi sửa một thuộc tính, bạn tạo **object mới**, sao chép các thuộc tính cũ rồi ghi đè thuộc tính cần đổi.

Công thức của bài

```
setCar((previous) => ({
  ...previous,
  brand: "Honda",
}));
```

Đang có	Thao tác	Sau khi cập nhật
brand: Toyota model: Vios year: "2024"	Đổi riêng brand thành Honda.	brand: Honda model: Vios year: "2024"

1. Tạo `src/CarEditor.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import CarEditor from "./CarEditor.jsx";

export default function App() {
  return <CarEditor />;
}
```

VÌ SAO CÓ ({ ... })?

Arrow function trả về object cần bọc object trong ngoặc tròn nếu viết ngắn. `prev => ({ ...prev })` trả object. Nếu viết `prev => { ... }`, JavaScript hiểu đó là thân hàm; bạn phải có return phù hợp.

Đọc thêm: R17, R30. Liên kết đầy đủ ở cuối tài liệu.

Form sửa thông tin xe

Tạo src/CarEditor.jsx. Năm đang giữ dạng chuỗi để dễ chỉnh ô nhập.

src/CarEditor.jsx - tạo file và dán toàn bộ

```
import { useState } from "react";

export default function CarEditor() {
  const [car, setCar] = useState({
    brand: "Toyota",
    model: "Vios",
    year: "2024",
  });

  function changeBrand(event) {
    const value = event.target.value;
    setCar((previous) => ({ ...previous, brand: value }));
  }

  function changeModel(event) {
    const value = event.target.value;
    setCar((previous) => ({ ...previous, model: value }));
  }

  function changeYear(event) {
    const value = event.target.value;
    setCar((previous) => ({ ...previous, year: value }));
  }

  return (
    <main className="panel">
      <h1>Thông tin xe</h1>
      <label>Hãng xe
        <input value={car.brand} onChange={changeBrand} />
      </label>
      <label>Mẫu xe
        <input value={car.model} onChange={changeModel} />
      </label>
      <label>Năm sản xuất
        <input type="number" value={car.year}
          onChange={changeYear} />
      </label>
      <p>{car.brand} - {car.model} - {car.year}</p>
    </main>
  );
}
```

Thay hãng thành Honda: dòng xem trước phải đổi hãng, còn mẫu và năm vẫn giữ nguyên. Tiếp tục sửa mẫu và năm để kiểm tra từng handler. Nếu mất các trường còn lại, bạn có thể đã thay cả object mà không spread dữ liệu cũ.

Đừng sửa trực tiếp object state

Phân biệt “sửa bản cũ” với “tạo bản mới từ bản cũ”.

Cách viết	Vấn đề / kết quả
`car.brand = "Honda"`	Sửa trực tiếp object state, không đi qua cơ chế cập nhật đúng.
`setCar({ brand: "Honda" })`	Thay cả object; model và year không được giữ tự động.
`{ ...previous, brand: value }`	Sao chép thuộc tính cũ, rồi ghi đè brand bằng value.
`{ brand: value, ...previous }`	Thuộc tính cũ nằm sau có thể ghi đè brand mới.

Object lồng nhau: phải sao chép từng tầng sửa

Minh họa với state student có object address

```
setStudent((previous) => ({
  ...previous,
  address: {
    ...previous.address,
    city: "TP.HCM",
  },
}));
```

Đoạn này giả sử student đã có address. Sao chép object ngoài chưa tạo bản sao address. Muốn đổi city mà vẫn giữ street, bạn cũng cần spread previous.address. Đây là ý nghĩa của “sao chép nông”.

Vì sao lấy value ra trước updater?

Trong CarEditor, handler đọc `event.target.value` vào biến value rồi mới gọi setter. Như vậy updater chỉ làm một việc: nhận car cũ, trả car mới. Bạn cũng dễ dùng debugger để xem giá trị vừa nhập.

TỰ LÀM TRONG 5-10 PHÚT

Thêm color với giá trị ban đầu "Đen" và một ô sửa màu.
Gợi ý: Thêm thuộc tính color, handler changeColor giống changeBrand, một input có value={car.color}, rồi hiển thị car.color.

Đọc thêm: R17, R30. Liên kết đầy đủ ở cuối tài liệu.

Cập nhật array trong state

Video 02:39:55 | Mảng state: thêm và xóa bằng cách tạo mảng mới.

Danh sách trái cây cần hai state: foods là các tên đã thêm; newFood là chữ đang nằm trong ô nhập. Đừng nhầm ô nhập với danh sách: gõ chữ chỉ đổi newFood, bấm Thêm mới đổi foods.

Mục đích	Cách làm
Thêm cuối mảng	`[...previous, name]`
Xóa theo tên trong demo	`previous.filter(food => food !== name)`
Duyệt để hiển thị	`foods.map(...)`

1. Tạo `src/FoodList.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import FoodList from "./FoodList.jsx";

export default function App() {
  return <FoodList />;
}
```

GIỚI HẠN CÓ CHỦ ĐÍCH CỦA BÀI NÀY

Demo chỉ lưu tên chuỗi và từ chối tên trùng khớp hoàn toàn, nên có thể dùng tên làm key. “Táo” và “táo” vẫn là hai chuỗi khác nhau. Khi cần cho phép trùng tên hoặc sửa tên, dùng object có id như bài 14.

`trim()` bỏ khoảng trắng ở hai đầu. `event.preventDefault()` ngăn hành vi gửi form mặc định làm tải lại trang. Dùng onSubmit trên form giúp cả bấm nút Thêm và nhấn Enter đều thêm được.

Đọc thêm: R18, R11. Liên kết đầy đủ ở cuối tài liệu.

Thêm và xóa trái cây

Tạo src/FoodList.jsx. Nút Xóa nằm ngoài form thêm nên không submit form.

src/FoodList.jsx - tạo file và dán toàn bộ

```
import { useState } from "react";

export default function FoodList() {
  const [foods, setFoods] = useState(["Táo", "Cam"]);
  const [newFood, setNewFood] = useState("");

  function addFood(event) {
    event.preventDefault();
    const name = newFood.trim();
    if (name === "" || foods.includes(name)) return;
    setFoods((previous) => [...previous, name]);
    setNewFood("");
  }

  function removeFood(name) {
    setFoods((previous) => previous.filter(
      (food) => food !== name
    ));
  }

  return (
    <main className="panel">
      <h1>Danh sách trái cây</h1>
      <form onSubmit={addFood} className="row">
        <input aria-label="Tên trái cây" value={newFood}
          onChange={(event) => setNewFood(event.target.value)} />
        <button type="submit">Thêm</button>
      </form>
      <p>Nhập tên khác chính xác các tên đang có.</p>
      <ul>
        {foods.map((food) => (
          <li key={food} className="row">
            <span>{food}</span>
            <button onClick={() => removeFood(food)}>Xóa</button>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Đọc kỹ thao tác thêm và xóa

Không cần nhớ nhiều hàm mảng; hiểu map, filter và spread trước.

Thêm “Xoài” diễn ra ra sao?

1. Người dùng nhập Xoài: newFood trở thành "Xoài"; foods chưa đổi.

2. Submit form: handler đọc và trim chữ; nếu rỗng hoặc trùng tên thì dừng.

3. Updater nhận foods hiện tại, tạo mảng mới có Xoài ở cuối.

4. setNewFood("") làm ô nhập trống lại. map tạo thêm một dòng mới.

Xóa “Táo” thực chất là giữ lại phần còn lại

Tư duy của filter

```
["Táo", "Cam", "Xoài"].filter((food) => food !== "Táo")
// Kết quả: ["Cam", "Xoài"]
```

Không nên làm trực tiếp với state	Dùng thay thế
`foods.push(name)`	`setFoods(prev => [...prev, name])`
`foods.splice(index, 1)`	`setFoods(prev => prev.filter(...))`
`foods.sort()`	Tạo bản sao bằng spread rồi mới sort.

Kết quả cần tự kiểm tra

- Ban đầu có Táo và Cam; thêm Xoài thì có ba dòng.
- Nhập toàn khoảng trắng: không thêm dòng rỗng.
- Nhập lại đúng chữ Táo: không tạo dòng trùng.
- Xóa Cam: Táo và Xoài còn nguyên; nhấn Enter vẫn thêm được.

TỰ LÀM TRONG 5-10 PHÚT

Thêm nút “Xóa tất cả” bên ngoài form.
Gợi ý: Dùng onClick={() => setFoods([])}. Dấu [] tạo một mảng rỗng mới.

KEY KHÁC VỊ TRÍ

Khi danh sách đổi, phần tử có thể chuyển vị trí. Ở app có thêm/xóa/sắp xếp, một id ổn định giúp React giữ đúng danh tính của dòng; vị trí index không bảo đảm điều đó.

Đọc thêm: R18, R10. Liên kết đầy đủ ở cuối tài liệu.

Cập nhật array chứa object

Video 02:48:55 | Mỗi phần tử của mảng giờ là một object có nhiều thuộc tính.

Bài 13 chỉ lưu "Táo", "Cam". Danh sách xe cần lưu cả id, hãng và mẫu. Cấu trúc là một mảng chứa object; cập nhật phải giữ đúng cả hai tầng: **mảng mới**, và **object mới cho xe cần sửa**.

Hình dạng dữ liệu

```
[
  { id: "c1", brand: "Toyota", model: "Vios" },
  { id: "c2", brand: "Honda", model: "City" },
]
```

Tác vụ	Kỹ thuật
Thêm xe	Tạo object mới, rồi spread vào mảng mới.
Xóa xe	filter, giữ những xe có id khác id cần xóa.
Sửa mẫu xe	map; xe đúng id thì trả object mới, xe khác trả lại object cũ.

1. Tạo `src/CarList.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import CarList from "./CarList.jsx";

export default function App() {
  return <CarList />;
}
```

ID PHÂN BIỆT CÁC XE

Hai xe có thể cùng hãng và cùng mẫu, nhưng vẫn có id khác nhau. Vì vậy xóa hoặc sửa dựa trên id sẽ chính xác hơn dựa trên tên hiển thị.

Đọc thêm: R18, R17, R33. Liên kết đầy đủ ở cuối tài liệu.

Danh sách xe: phần xử lý

src/CarList.jsx - phần 1/2. Nối phần giao diện ở trang sau.

src/CarList.jsx - phần 1/2

```
import { useState } from "react";

export default function CarList() {
  const [cars, setCars] = useState([
    { id: "c1", brand: "Toyota", model: "Vios" },
  ]);
  const [brand, setBrand] = useState("");
  const [model, setModel] = useState("");

  function addCar(event) {
    event.preventDefault();
    if (!brand.trim() || !model.trim()) return;
    const newCar = {
      id: crypto.randomUUID(),
      brand: brand.trim(),
      model: model.trim(),
    };
    setCars((previous) => [...previous, newCar]);
    setBrand("");
    setModel("");
  }

  function renameModel(id, value) {
    setCars((previous) => previous.map((car) => (
      car.id === id ? { ...car, model: value } : car
    )));
  }

  function removeCar(id) {
    setCars((previous) => previous.filter(
      (car) => car.id !== id
    ));
  }
```

newCar được tạo trong handler, bên ngoài updater. brand và model trên form là dữ liệu đang nhập; car.brand và car.model trong danh sách là dữ liệu đã được thêm. Sau khi thêm, hai ô form được xóa rỗng để nhập xe tiếp theo.

Đọc thêm: R18, R33. Liên kết đầy đủ ở cuối tài liệu.

Danh sách xe: phần giao diện

Nối ngay sau phần 1, không lặp lại import hay khai báo function.

src/CarList.jsx - phần 2/2

```
  return (
    <main className="panel">
      <h1>Danh sách xe</h1>
      <form className="row" onSubmit={addCar}>
        <input aria-label="Hãng xe" placeholder="Hãng xe"
          value={brand} onChange={(event) => {
            setBrand(event.target.value);
          }} />
        <input aria-label="Mẫu xe" placeholder="Mẫu xe"
          value={model} onChange={(event) => {
            setModel(event.target.value);
          }} />
        <button type="submit">Thêm xe</button>
      </form>
      <ul>
        {cars.map((car) => (
          <li key={car.id} className="row">
            <strong>{car.brand}</strong>
            <input aria-label={`Mẫu xe ${car.brand}`}
              value={car.model} onChange={(event) => {
                renameModel(car.id, event.target.value);
              }} />
            <button onClick={() => removeCar(car.id)}>
              Xóa
            </button>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Mỗi dòng có một input sửa mẫu xe. Khi bạn gõ ở dòng nào, handler gửi id của đúng xe đó và giá trị mới cho renameModel. Nút Xóa cũng truyền đúng id của dòng đang bấm.

Vì sao sửa một xe mà xe khác không đổi?

Lần theo một lời gọi: renameModel("c1", "Corolla").

Phần cốt lõi, đã có trong CarList.jsx

```
setCars((previous) => previous.map((car) => (
  car.id === id ? { ...car, model: value } : car
)));
```

1. map đi qua từng xe trong mảng cũ.

2. Nếu car.id bằng id cần sửa, tạo object mới bằng spread và ghi đè model.

3. Nếu id khác, trả lại car. Không cần sao chép những object không sửa.

4. map trả mảng mới; React nhận state mới và tính lại giao diện.

Bạn thao tác	Kết quả cần thấy
Thêm Honda / City	Xuất hiện xe mới; form thêm trở về rỗng.
Thêm thêm một Honda / City	Hai dòng riêng biệt cùng tên, id khác nhau.
Sửa mẫu ở một dòng	Chỉ dòng ấy thay đổi.
Xóa một dòng	Các dòng khác vẫn giữ đúng nội dung.

SPREAD MẢNG CHƯA ĐỦ KHI SỬA OBJECT BÊN TRONG

`const next = [...cars]; next[0].model = "X";` vẫn sửa object được chia sẻ với mảng cũ. Cách đúng là thay object cần sửa bằng một object mới như code trên.

TỰ LÀM TRONG 5-10 PHÚT

Thêm thuộc tính year cho mỗi xe, với ô nhập năm khi thêm.
Gợi ý: Tạo state year cho form; thêm year vào newCar và dữ liệu khởi tạo; hiện car.year trên mỗi dòng. Có thể giữ chuỗi lúc nhập.

Đọc thêm: R18, R17. Liên kết đầy đủ ở cuối tài liệu.

Ứng dụng To-do List

Video 03:01:42 | Kết hợp những gì đã học để làm ứng dụng hoàn chỉnh đầu tiên.

To-do list này có nhập việc mới, thêm bằng Enter, xóa, đánh dấu hoàn thành và chuyển việc lên/xuống. Bản demo dùng id ổn định để các thao tác vẫn đúng khi đổi thứ tự; giao diện chỉ dùng CSS đơn giản.

Dữ liệu	Lý do cần
`tasks`	Mảng object { id, text, done } cho các công việc.
`newTask`	Chữ đang gõ vào ô nhập.
`remaining`	Số việc chưa xong, tính từ tasks; không cần state riêng.

1. Tạo `src/TodoList.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import TodoList from "./TodoList.jsx";

export default function App() {
  return <TodoList />;
}
```

Thứ tự làm để không bị ngợp

1. Dán đầy đủ phần 1 và phần 2 của TodoList.jsx ở hai trang tiếp theo.

2. Chạy được thêm và xóa trước. Sau đó thử tick hoàn thành.

3. Cuối cùng thử Lên/Xuống; thêm CSS ở trang giải thích giao diện.

TẠI SAO KHÔNG DÙNG INDEX LÀM KEY?

Danh sách này có xóa và đổi thứ tự, nên vị trí một việc không cố định. key={task.id} giữ danh tính công việc. index vẫn hữu ích để biết dòng đang ở đầu hay cuối và vô hiệu hóa nút tương ứng.

Đọc thêm: R10, R18. Liên kết đầy đủ ở cuối tài liệu.

To-do List: dữ liệu và các hàm

src/TodoList.jsx - phần 1/2. Dán tiếp phần JSX ở trang sau vào cùng file.

src/TodoList.jsx - phần 1/2

```
import { useState } from "react";

export default function TodoList() {
  const [tasks, setTasks] = useState([
    { id: "t1", text: "Học React", done: false },
    { id: "t2", text: "Làm bài tập", done: false },
  ]);
  const [newTask, setNewTask] = useState("");

  function addTask(event) {
    event.preventDefault();
    const text = newTask.trim();
    if (text === "") return;
    const task = { id: crypto.randomUUID(), text, done: false };
    setTasks((previous) => [...previous, task]);
    setNewTask("");
  }

  function deleteTask(id) {
    setTasks((previous) => previous.filter(
      (task) => task.id !== id
    ));
  }

  function toggleTask(id) {
    setTasks((previous) => previous.map((task) => (
      task.id === id ? { ...task, done: !task.done } : task
    )));
  }

  function moveTask(id, direction) {
    setTasks((previous) => {
      const index = previous.findIndex((task) => task.id === id);
      const target = index + direction;
      if (index < 0 || target < 0 || target >= previous.length) {
        return previous;
      }
      const next = [...previous];
      [next[index], next[target]] = [next[target], next[index]];
      return next;
    });
  }

  const remaining = tasks.filter((task) => !task.done).length;
```

To-do List: giao diện

Phần 2/2 - nối ngay sau phần 1 rồi lưu file.

src/TodoList.jsx - phần 2/2

```
  return (
    <main className="panel">
      <h1>To-do List</h1>
      <form onSubmit={addTask} className="row">
        <input aria-label="Việc mới" placeholder="Nhập việc mới"
          value={newTask} onChange={(event) => {
            setNewTask(event.target.value);
          }} />
        <button type="submit">Thêm</button>
      </form>
      <p>Còn {remaining} việc chưa xong.</p>
      {tasks.length === 0 && <p>Chưa có công việc nào.</p>}
      <ul className="task-list">
        {tasks.map((task, index) => (
          <li key={task.id} className="task">
            <label className="task-text">
              <input type="checkbox" checked={task.done}
                onChange={() => toggleTask(task.id)} />
              <span className={task.done ? "done" : ""}>
                {task.text}
              </span>
            </label>
            <div className="row">
              <button disabled={index === 0}
                onClick={() => moveTask(task.id, -1)}>Lên</button>
              <button disabled={index === tasks.length - 1}
                onClick={() => moveTask(task.id, 1)}>Xuống</button>
              <button onClick={() => deleteTask(task.id)}>Xóa</button>
            </div>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

CSS và cách đọc To-do List

Thêm CSS này vào cuối index.css đã có CSS nền của bài 01.

src/index.css - thêm ở cuối, giữ CSS nền ở bài 01

```
.task-list { list-style: none; padding: 0; }
.task {
  display: flex; flex-wrap: wrap; align-items: center;
  justify-content: space-between; gap: 12px;
  padding: 12px; border: 1px solid #dbe2ea; border-radius: 8px;
}
.task-text { display: flex; align-items: center; gap: 8px; margin: 0; }
.done { text-decoration: line-through; color: #64748b; }
```

Hiểu từng thao tác

Hàm / giá trị	Cách hoạt động
addTask	trim chữ, bỏ qua nếu rỗng, tạo id, thêm object mới, xóa ô nhập.
deleteTask	filter giữ các task có id khác id cần xóa.
toggleTask	map và spread object; đổi done thành !done ở task đúng id.
remaining	filter các task chưa done rồi đọc length.

Chữ gạch ngang đến từ đâu?

Khi task.done là true, span nhận className="done". CSS text-decoration: line-through tạo gạch ngang. Đây là state điều khiển class CSS, không cần trực tiếp tìm phần tử rồi sửa style của nó.

TRÁNH ĐẶT LỆNH THAY STATE NGAY TRONG JSX

Viết `onClick={() => deleteTask(task.id)}`. Nếu viết `onClick={deleteTask(task.id)}`, hàm chạy trong lúc render thay vì đợi bấm; nó có thể gây vòng lặp render hoặc xóa dữ liệu ngay khi mở trang.

TỰ LÀM TRONG 5-10 PHÚT

Thêm nút “Xóa việc đã xong”.
Gợi ý: setTasks(previous => previous.filter(task => !task.done)). Đặt nút bên ngoài form thêm việc.

Hiểu đổi thứ tự và tự kiểm tra

moveTask nhận id của việc và direction: -1 để lên, 1 để xuống.

1. findIndex tìm vị trí hiện tại của việc trong mảng mới nhất.

2. target là vị trí mới. Nếu vượt đầu/cuối hoặc không tìm thấy id, trả lại mảng cũ.

3. Spread tạo mảng next mới. Đổi chỗ hai phần tử trong bản sao.

4. Trả next cho React. Bạn không sửa mảng state cũ.

Ví dụ riêng để hiểu phép đổi chỗ

```
const next = ["A", "B", "C"];
const index = 1;  // B đang ở vị trí 1
const target = 0; // Muốn B đi lên vị trí 0
[next[index], next[target]] = [next[target], next[index]];
// next trở thành ["B", "A", "C"]
```

Thử theo thứ tự	Kết quả đúng
Thêm “Ôn Java”	Có ba việc; ô nhập rỗng trở lại.
Tick hoàn thành việc đầu	Chữ gạch ngang; số chưa xong giảm 1.
Chuyển việc đã tick xuống	Dấu tick và chữ gạch ngang đi cùng đúng công việc.
Bấm Lên ở đầu / Xuống ở cuối	Nút bị vô hiệu hóa, thứ tự không vượt giới hạn.
Xóa tất cả từng việc	Hiện “Chưa có công việc nào.”
Thêm toàn dấu cách	Không tạo việc mới.
F5 tải lại trang	Trở lại hai việc mẫu; dữ liệu chưa lưu lâu dài.

CHƯA CẦN THÊM LOCALSTORAGE NGAY

Trước hết hãy hiểu được thêm, sửa, xóa và đổi thứ tự. Lưu lâu dài là bước mở rộng khác; useState không tự lưu sau khi đóng hoặc tải lại trang.

Đọc thêm: R18, R10. Liên kết đầy đủ ở cuối tài liệu.

Hook useEffect

Video 03:24:17 | useEffect giúp đồng bộ component với hệ thống bên ngoài React.

Hiểu trước khi gõ

Giao diện thường được tính từ props và state. Nhưng có việc nằm ngoài phép tính JSX: đổi tiêu đề tab, chạy timer hoặc đăng ký sự kiện của window. Effect là nơi thiết lập và dọn dẹp những kết nối kiểu đó sau khi React commit giao diện.

Hình dạng của một Effect - minh họa cấu trúc

```
useEffect(() => {
  // Thiết lập việc cần đồng bộ

  return () => {
    // Dọn dẹp, nếu việc thiết lập cần được hủy
  };
}, [dependency]);
```

Đối số thứ hai	Khi nào Effect được thiết lập lại?
Không truyền	Sau mỗi lần component commit.
`[]`	Sau khi component được gắn vào cây; không lặp vì thay đổi props/state. Có thể chạy lại khi remount và có kiểm tra thêm trong StrictMode lúc phát triển.
`[count]`	Lần đầu và sau commit khi count thay đổi so với lần trước.

Commit là giai đoạn React áp dụng thay đổi cần thiết vào DOM. Không nên hiểu useEffect đơn giản là “luôn chạy sau khi trình duyệt đã vẽ xong màn hình”; thời điểm so với việc vẽ còn tùy tình huống. Trong các bài này, bạn chỉ cần phân biệt render với công việc đồng bộ ở Effect.

DEPENDENCIES KHÔNG PHẢI DANH SÁCH CHỌN TÙY THÍCH

Hãy khai báo các giá trị reactive mà Effect đọc, như props và state. Đừng xóa dependency chỉ để làm mất cảnh báo. Một mảng [] sai có thể khiến Effect giữ dữ liệu cũ.

Đọc thêm: R19, R20, R32. Liên kết đầy đủ ở cuối tài liệu.

Demo: đổi tiêu đề tab

Tạo EffectDemo.jsx, rồi thay App.jsx theo đoạn dưới.

src/EffectDemo.jsx - tạo file và dán toàn bộ

```
import { useEffect, useState } from "react";

export default function EffectDemo() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const previousTitle = document.title;
    document.title = `Bạn đã bấm ${count} lần`;

    return () => {
      document.title = previousTitle;
    };
  }, [count]);

  return (
    <main className="panel">
      <h1>Đồng bộ tiêu đề tab</h1>
      <p>Số lần bấm: {count}</p>
      <button onClick={() => setCount((value) => value + 1)}>
        Tăng và nhìn lên tiêu đề tab
      </button>
    </main>
  );
}
```

src/App.jsx - thay toàn bộ nội dung

```
import EffectDemo from "./EffectDemo.jsx";

export default function App() {
  return <EffectDemo />;
}
```

Tutorial từng bước

1. Mở trang: nhìn chữ trên tab trình duyệt, không chỉ phần nội dung trang.

2. Bấm nút: số trên giao diện và tiêu đề tab tăng cùng nhau.

3. Effect đọc count nên dependency là [count].

4. Cleanup phục hồi tiêu đề trước đó; trước lần setup mới, React dọn Effect cũ rồi đồng bộ bằng count mới.

Đọc thêm: R19. Liên kết đầy đủ ở cuối tài liệu.

Cleanup: dựng lên thì biết dọn đi

Ví dụ bổ sung độc lập: lắng nghe sự kiện đổi kích thước cửa sổ.

src/WindowSize.jsx - tùy chọn, một file hoàn chỉnh

```
import { useEffect, useState } from "react";

export default function WindowSize() {
  const [width, setWidth] = useState(() => window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);

    return () => {
      window.removeEventListener("resize", handleResize);
    };
  }, []);

  return (
    <main className="panel">
      <h1>Chiều rộng cửa sổ</h1>
      <p>{width} px</p>
    </main>
  );
}
```

Muốn chạy ví dụ phụ này, đổi App.jsx thành import WindowSize from "./WindowSize.jsx" và return <WindowSize />. Kéo hẹp/rộng cửa sổ: số px thay đổi. Demo này dành cho project client chạy trong trình duyệt như Vite của bài 01.

Lúc thiết lập	Lúc dọn dẹp
`addEventListener`	`removeEventListener` với cùng hàm handler.
`setInterval`	`clearInterval` với đúng ID đã tạo.
Mở kết nối bên ngoài	Đóng kết nối tương ứng nếu API yêu cầu.

CLEANUP KHÔNG CHỈ CHẠY KHI RỜI TRANG

Nó chạy trước setup mới khi dependencies đổi, và khi component unmount. Trong StrictMode của môi trường phát triển, React còn thử thêm một vòng setup/cleanup để phát hiện lỗi.

Đọc thêm: R19, R20, R22. Liên kết đầy đủ ở cuối tài liệu.

Đừng biến mọi việc thành Effect

Phân biệt dữ liệu suy ra, thao tác người dùng và đồng bộ bên ngoài.

Việc cần làm	Đặt ở đâu?
Tính tổng tiền từ số lượng	Tính trực tiếp trong component.
Đếm việc chưa hoàn thành	Tính từ tasks bằng filter; không cần state phụ.
Xóa việc sau khi bấm Xóa	Trong handler onClick.
Bắt đầu/hủy một timer theo state	Effect có cleanup.
Đăng ký/hủy sự kiện window	Effect có cleanup.

Một vòng lặp dễ vô tình tạo ra

Ví dụ sai - đừng thêm vào demo

```
useEffect(() => {
  setCount(count + 1);
}, [count]);
```

Effect đổi count; count đổi lại làm Effect chạy; Effect lại đổi count. Đây là vòng lặp. Nếu mục tiêu là tăng khi bấm, đặt setter trong onClick. Nếu mục tiêu là timer, quản lý timer và cleanup như bài đồng hồ.

Vì sao Console in hai lần khi mới mở?

Với StrictMode ở gốc như main.jsx của tài liệu, môi trường phát triển có các lần gọi kiểm tra thêm. Hãy kiểm tra code thuần và cleanup đúng, đừng chỉ xóa StrictMode để che triệu chứng. Handler bấm nút không bị StrictMode cố ý gọi hai lần theo cách này.

TỰ LÀM TRONG 5-10 PHÚT

Trong EffectDemo, thêm nút đặt count về 0.
Gợi ý: Dùng setCount(0) trong handler. Effect đang có [count] nên sẽ tự đồng bộ tiêu đề khi count thực sự đổi.

Đọc thêm: R21, R22, R19. Liên kết đầy đủ ở cuối tài liệu.

Đồng hồ điện tử

Video 03:44:08 | Đồng hồ điện tử = thời gian hiện tại + cập nhật định kỳ.

Đồng hồ phải tiếp tục đổi dù bạn không bấm nút. Vì vậy nó dùng timer. State giữ một Date để render; Effect tạo interval khi component được gắn vào cây, và cleanup hủy interval khi component rời cây.

1. Tạo `src/DigitalClock.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import DigitalClock from "./DigitalClock.jsx";

export default function App() {
  return <DigitalClock />;
}
```

src/index.css - thêm ở cuối, giữ CSS nền ở bài 01

```
.clock {
  font-size: clamp(32px, 8vw, 64px);
  font-family: monospace; font-variant-numeric: tabular-nums;
  text-align: center; margin: 20px 0;
}
```

Thành phần	Nhiệm vụ
`new Date()`	Đọc thời điểm hiện tại.
`setInterval(..., 1000)`	Yêu cầu callback chạy lặp với khoảng chờ 1 giây.
`setTime(new Date())`	Lấy giờ hiện tại mới, yêu cầu render lại.
`clearInterval(intervalId)`	Dừng timer đã tạo.

Đọc thêm: R19, R23, R24. Liên kết đầy đủ ở cuối tài liệu.

DigitalClock hoàn chỉnh

Kết quả: định dạng 24 giờ HH:MM:SS, cập nhật theo giờ trên máy bạn.

src/DigitalClock.jsx - tạo file và dán toàn bộ

```
import { useEffect, useState } from "react";

function formatTime(date) {
  const hours = String(date.getHours()).padStart(2, "0");
  const minutes = String(date.getMinutes()).padStart(2, "0");
  const seconds = String(date.getSeconds()).padStart(2, "0");
  return `${hours}:${minutes}:${seconds}`;
}

export default function DigitalClock() {
  const [time, setTime] = useState(() => new Date());

  useEffect(() => {
    const intervalId = setInterval(() => {
      setTime(new Date());
    }, 1000);

    return () => clearInterval(intervalId);
  }, []);

  return (
    <main className="panel">
      <h1>Đồng hồ điện tử</h1>
      <p className="clock">{formatTime(time)}</p>
      <p>Giờ địa phương trên thiết bị của bạn.</p>
    </main>
  );
}
```

Các phép biến đổi nhỏ

getHours lấy giờ 0-23, getMinutes và getSeconds lấy phút/giây. String biến số thành chuỗi. padStart(2, "0") thêm số 0 vào bên trái nếu chưa đủ hai ký tự: 7 thành "07"; 15 vẫn là "15".

LAZY INITIALIZER

useState(() => new Date()) truyền một hàm để tạo giá trị ban đầu khi khởi tạo state. Những lần render sau không cần gọi hàm khởi tạo này chỉ để bỏ qua kết quả; trong StrictMode, React có thể gọi thêm để kiểm tra lúc phát triển.

Đọc thêm: R13, R23. Liên kết đầy đủ ở cuối tài liệu.

Đồng hồ chạy đúng ý nghĩa

Interval là cơ chế gọi lại; thời gian thật được đọc từ Date.

Vì sao không chỉ cộng thêm một giây?

Timer của trình duyệt không đảm bảo callback luôn tới đúng từng mốc 1000 ms, nhất là khi tab chạy nền hoặc máy bận. Lấy new Date() mỗi lần giúp đồng hồ bắt lại giờ hiện tại, thay vì cộng dồn sai lệch theo số callback.

Kịch bản để tự kiểm tra

1. Mở trang: giờ hiện ngay, không cần đợi một giây để có giá trị đầu.

2. Để yên vài giây: phần giây thay đổi, phút chuyển khi đủ 60 giây.

3. Đổi App sang một demo khác: timer của DigitalClock phải được cleanup khi component không còn xuất hiện.

4. Quay lại demo: nó lấy giờ hiện tại, không tiếp tục một bộ đếm cũ.

Muốn định dạng theo múi giờ Việt Nam - thay dòng hiển thị giờ

```
<p className="clock">
  {time.toLocaleTimeString("vi-VN", {
    timeZone: "Asia/Ho_Chi_Minh",
    hour12: false,
  })}
</p>
```

Nếu dùng đoạn thay thế, xóa hàm formatTime cũ nếu nó không còn được gọi. Dạng hiển thị cụ thể của toLocaleTimeString do bộ định dạng locale của môi trường quyết định.

TỰ LÀM TRONG 5-10 PHÚT

Thêm ngày hiện tại dưới giờ bằng time.toLocaleDateString("vi-VN").
Gợi ý: Thêm {time.toLocaleDateString("vi-VN")} . Bạn có sẵn state time, nên không cần một state date thứ hai.

KHÔNG TẠO INTERVAL NGAY TRONG THÂN COMPONENT

Nếu đặt setInterval trực tiếp trước return, mỗi lần render có thể tạo thêm một timer. Đặt nó trong Effect và trả cleanup sẽ giúp vòng đời timer rõ ràng.

Đọc thêm: R23, R24, R20. Liên kết đầy đủ ở cuối tài liệu.

Hook useContext

Video 04:00:08 | Context giúp component ở sâu nhận dữ liệu mà không truyền props qua từng tầng.

Giả sử ContextDemo giữ tên người dùng. Nó hiển thị Dashboard, Dashboard lại hiển thị UserPanel. UserPanel cần tên; Dashboard thì không dùng tên. Context cho phép UserPanel đọc dữ liệu từ Provider phía trên mà Dashboard không cần nhận và chuyển tiếp prop ấy.

Vị trí	Vai trò trong demo
ContextDemo	Giữ state name, cung cấp { name, setName }.
Dashboard	Tầng trung gian, chỉ bố trí giao diện.
UserPanel	Đọc context và bấm nút đổi tên.

1. Tạo `src/ContextDemo.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import ContextDemo from "./ContextDemo.jsx";

export default function App() {
  return <ContextDemo />;
}
```

Ba bước cần nhớ

1. createContext tạo một context ở ngoài các component.

2. Provider bọc vùng component được phép nhận giá trị.

3. useContext đọc giá trị từ Provider gần nhất nằm phía trên.

CONTEXT KHÔNG THAY THẾ STATE

Trong demo, dữ liệu vẫn được lưu bằng useState. Context là cách cung cấp dữ liệu và hàm cập nhật cho những component con cần nó. Nó không tự ghi dữ liệu vào database hay giữ tên qua F5.

Đọc thêm: R25, R26. Liên kết đầy đủ ở cuối tài liệu.

Một file để dễ nhìn toàn bộ luồng

Tạo src/ContextDemo.jsx. Các component phụ đặt ngoài ContextDemo.

src/ContextDemo.jsx - tạo file và dán toàn bộ

```
import { createContext, useContext, useState } from "react";

const UserContext = createContext(null);

function Dashboard() {
  return (
    <section>
      <h2>Khu vực học tập</h2>
      <UserPanel />
    </section>
  );
}

function UserPanel() {
  const { name, setName } = useContext(UserContext);

  return (
    <div>
      <p>Xin chào, {name}!</p>
      <button onClick={() => setName("Hoàng Vũ")}>
        Đổi tên trong context
      </button>
    </div>
  );
}

export default function ContextDemo() {
  const [name, setName] = useState("Vũ");

  return (
    <UserContext.Provider value={{ name, setName }}>
      <main className="panel">
        <h1>Tài khoản: {name}</h1>
        <Dashboard />
      </main>
    </UserContext.Provider>
  );
}
```

Ban đầu tiêu đề tài khoản và lời chào đều có Vũ. Bấm “Đổi tên trong context”: cả hai cùng đổi thành Hoàng Vũ, dù Dashboard không hề có prop name.

Hiểu Provider và nơi đọc context

Chỉ cần props nếu dữ liệu truyền xuống một vài tầng rõ ràng.

Provider gần nhất có nghĩa gì?

Nếu có nhiều Provider của cùng context lồng nhau, component đọc giá trị từ Provider gần nhất ở phía trên nó. Provider không tự có hiệu lực đối với một useContext được gọi trong chính component đang tạo Provider đó.

Giá trị mặc định null

createContext(null) nghĩa là không có Provider phù hợp thì useContext nhận null. Dòng tách `{ name, setName }` trong UserPanel sẽ lỗi nếu bạn mang UserPanel ra ngoài Provider. Đây là lý do cấu trúc bọc component phải đúng.

Tình huống	Ưu tiên
Một Card nhận tên từ App	Props, vì quan hệ ngắn và rõ.
Nhiều nhánh cần theme hoặc người dùng	Cân nhắc Context.
Ô nhập chỉ dùng trong một form	State local trong form.

Cú pháp Provider trong React 19

Ví dụ dùng `<UserContext.Provider value={...}>` để dễ đối chiếu với các hướng dẫn React trước đây. Cách đó vẫn dùng được trong React 19. React 19 cũng hỗ trợ dạng ngắn `<UserContext value={...}>`; ở đây cứ giữ một kiểu nhất quán.

TỰ LÀM TRONG 5-10 PHÚT

Thay nút đổi tên trong UserPanel bằng một input.
Gợi ý: value={name}; onChange={(event) => setName(event.target.value)}. Tiêu đề ở ContextDemo cũng phải đổi theo lúc bạn gõ.

ĐỪNG TẠO CONTEXT MỚI TRONG MỖI LẦN RENDER

Đặt createContext bên ngoài component như trong demo. Khi project lớn hơn, có thể tách nó vào file riêng và import cùng một context ở nơi cung cấp và nơi đọc.

Đọc thêm: R25, R26. Liên kết đầy đủ ở cuối tài liệu.

Hook useRef

Video 04:11:44 | useRef giữ một giá trị qua các lần render nhưng đổi nó không tự yêu cầu render.

Có dữ liệu bạn cần nhớ để xử lý, nhưng không cần hiển thị liên tục: ID timer, mốc bắt đầu, hay một phần tử DOM để focus. useRef tạo object có thuộc tính `.current` cho những tình huống đó.

Đặc điểm	useState	useRef
Đọc giá trị	Đọc biến state.	Đọc ref.current trong handler hoặc Effect.
Cập nhật	Gọi setter.	Gán ref.current.
Tự yêu cầu render	Có thể, khi giá trị đổi.	Không.
Ví dụ phù hợp	Chữ đang gõ, số trên màn hình.	Tham chiếu input, mốc thời gian.

1. Tạo `src/RefDemo.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import RefDemo from "./RefDemo.jsx";

export default function App() {
  return <RefDemo />;
}
```

KHÔNG DÙNG REF ĐỂ NÉ STATE

Nếu bạn muốn màn hình tự đổi theo dữ liệu, thường cần state. Không đọc/ghi ref.current trong quá trình render để điều khiển giao diện. Demo sẽ đọc ref trong handler rồi chép vào state khi cần hiện lên.

Đọc thêm: R27, R28. Liên kết đầy đủ ở cuối tài liệu.

Demo focus input và nhớ số lần bấm

Tạo src/RefDemo.jsx. Ô nhập này không có value prop: đó là uncontrolled input.

src/RefDemo.jsx - tạo file và dán toàn bộ

```
import { useRef, useState } from "react";

export default function RefDemo() {
  const inputRef = useRef(null);
  const clickRef = useRef(0);
  const [shownClicks, setShownClicks] = useState(0);

  function focusInput() {
    if (inputRef.current) inputRef.current.focus();
  }

  function countSilently() {
    clickRef.current += 1;
    console.log("Giá trị ref:", clickRef.current);
  }

  return (
    <main className="panel">
      <h1>Thử useRef</h1>
      <label>Họ tên
        <input ref={inputRef} placeholder="Nhập tên" />
      </label>
      <div className="row">
        <button onClick={focusInput}>Đưa con trỏ vào ô</button>
        <button onClick={countSilently}>Tăng ref</button>
        <button onClick={() => setShownClicks(clickRef.current)}>
          Hiện giá trị ref
        </button>
      </div>
      <p>Giá trị đã chụp từ ref: {shownClicks}</p>
    </main>
  );
}
```

Input trong demo này do DOM giữ nội dung; React chỉ giữ tham chiếu đến nó để gọi focus. Việc không có value/onChange ở đây là có chủ đích, khác với form controlled ở bài 09.

Đọc thêm: R27, R15. Liên kết đầy đủ ở cuối tài liệu.

Tự chứng minh ref không tự render

Làm đúng thứ tự để thấy sự khác biệt rõ nhất.

1. Mở trang: “Giá trị đã chụp từ ref” đang là 0.

2. Bấm “Tăng ref” ba lần. Console lần lượt in 1, 2, 3; dòng trên màn hình vẫn là 0.

3. Bấm “Hiện giá trị ref”. Handler đọc clickRef.current và gọi setShownClicks; màn hình thành 3.

4. Bấm “Đưa con trỏ vào ô”: input được focus và có thể gõ ngay.

Tên trong demo	Nó nhớ điều gì?
inputRef	Phần tử input sau khi React gắn nó vào DOM.
clickRef	Số lần đã bấm nút Tăng ref.
shownClicks	Bản chụp số lần bấm tại lúc nhấn Hiện giá trị ref.

Vì sao phải kiểm tra inputRef.current?

Ban đầu nó là null. React gán phần tử DOM sau khi gắn vào giao diện, và đặt lại null khi gỡ phần tử. Guard `if (inputRef.current)` giúp handler không gọi focus trên null.

Biến thường, ref và state

Biến trong thân component được tạo lại ở mỗi lần gọi. Ref được React giữ lại giữa các lần render. State cũng được giữ, và có cơ chế yêu cầu giao diện cập nhật. Chọn theo vai trò dữ liệu, không theo việc cách nào ít dòng hơn.

TỰ LÀM TRONG 5-10 PHÚT

Thêm nút “Reset ref” để đưa clickRef về 0 và cập nhật cả dòng hiển thị.
Gợi ý: Trong handler: clickRef.current = 0; setShownClicks(0). Nếu chỉ gán ref, phần hiển thị vẫn giữ số cũ.

Đọc thêm: R27, R28. Liên kết đầy đủ ở cuối tài liệu.

Đồng hồ bấm giờ

Video 04:23:01 | Ghép useState, useEffect và useRef vào một ứng dụng có nhiều trạng thái.

Đồng hồ bấm giờ cần Bắt đầu, Dừng, Chạy tiếp và Đặt lại. Khác đồng hồ điện tử, nó hiển thị **thời gian đã trôi qua**, không phải giờ hiện tại trong ngày.

Dữ liệu	Dùng gì?	Lý do
elapsed	State	Số thời gian cần hiển thị trên màn hình.
isRunning	State	Điều khiển trạng thái nút và Effect tạo timer.
startRef	Ref	Mốc bắt đầu tính chênh lệch.
savedRef	Ref	Số thời gian đã tích lũy lúc dừng.
runningRef	Ref	Cờ kiểm tra ngay trong handler/callback để chặn lệnh lặp.

1. Tạo `src/Stopwatch.jsx`. Chép code của bài vào đúng file này.

2. Thay toàn bộ `src/App.jsx` bằng đoạn nối component của bài. Giữ `main.jsx` và CSS nền của bài 01.

3. Lưu các file. Nếu chưa chạy server, mở Terminal tại thư mục có `package.json` và chạy `npm run dev`.

src/App.jsx - thay toàn bộ nội dung

```
import Stopwatch from "./Stopwatch.jsx";

export default function App() {
  return <Stopwatch />;
}
```

> **CODE CHIA BA PHẦN LIÊN TIẾP**
>
> Ghép phần 1, phần 2 rồi phần 3 vào cùng file Stopwatch.jsx. Không thêm dấu đóng hàm riêng sau từng phần. Sau khi có đủ file, thêm CSS và chạy kiểm tra theo trang cuối bài.

Đọc thêm: R19, R27, R29. Liên kết đầy đủ ở cuối tài liệu.

Stopwatch: định dạng và timer

src/Stopwatch.jsx - phần 1/3. Timer cập nhật giao diện, không quyết định số mili giây đã trôi qua.

src/Stopwatch.jsx - phần 1/3

```
import { useEffect, useRef, useState } from "react";

function formatElapsed(milliseconds) {
  const totalSeconds = Math.floor(milliseconds / 1000);
  const minutes = Math.floor(totalSeconds / 60);
  const seconds = totalSeconds % 60;
  const hundredths = Math.floor(milliseconds / 10) % 100;
  return [minutes, seconds, hundredths]
    .map((value) => String(value).padStart(2, "0"))
    .join(":");
}

export default function Stopwatch() {
  const [elapsed, setElapsed] = useState(0);
  const [isRunning, setIsRunning] = useState(false);
  const startRef = useRef(0);
  const savedRef = useRef(0);
  const runningRef = useRef(false);

  useEffect(() => {
    if (!isRunning) return;
    const intervalId = setInterval(() => {
      if (!runningRef.current) return;
      setElapsed(performance.now() - startRef.current);
    }, 10);
    return () => clearInterval(intervalId);
  }, [isRunning]);
```

performance.now() cung cấp mốc thời gian theo mili giây. Lấy mốc hiện tại trừ startRef.current để tính elapsed. Mốc này không phải giờ trong ngày; nó phù hợp để đo chênh lệch thời gian trong phiên đang chạy.

Callback kiểm tra runningRef trước khi cập nhật, giúp tránh một lượt timer còn chờ làm đổi số sau thao tác dừng/reset. Cleanup hủy interval khi isRunning đổi hoặc component unmount.

Đọc thêm: R29, R24. Liên kết đầy đủ ở cuối tài liệu.

Stopwatch: bắt đầu, dừng, đặt lại

src/Stopwatch.jsx - phần 2/3. Nối đoạn này ngay sau phần 1.

src/Stopwatch.jsx - phần 2/3

```
  function start() {
    if (runningRef.current) return;
    runningRef.current = true;
    startRef.current = performance.now() - savedRef.current;
    setIsRunning(true);
  }

  function stop() {
    if (!runningRef.current) return;
    runningRef.current = false;
    savedRef.current = performance.now() - startRef.current;
    setElapsed(savedRef.current);
    setIsRunning(false);
  }

  function reset() {
    runningRef.current = false;
    savedRef.current = 0;
    startRef.current = 0;
    setIsRunning(false);
    setElapsed(0);
  }
```

Chạy tiếp không bắt đầu lại từ 0

Giả sử đã chạy 3.500 ms rồi dừng. Lúc bắt đầu lại, performance.now() là 10.000 ms. Ta đặt startRef thành 10.000 - 3.500 = 6.500. Khi mốc hiện tại là 10.600, elapsed bằng 10.600 - 6.500 = 4.100 ms. Như vậy đã cộng đúng 600 ms mới vào 3.500 ms cũ.

HAI GIÁ TRỊ “ĐANG CHẠY” CÓ VAI TRÒ RIÊNG

isRunning phục vụ render và Effect. runningRef được cập nhật ngay trong handler và đọc trong callback timer. Trong demo này, hai giá trị luôn được đổi cùng nhau bởi start, stop và reset; không sửa riêng một bên.

Stopwatch: giao diện và CSS

src/Stopwatch.jsx - phần 3/3. Nối sau phần 2 rồi lưu.

src/Stopwatch.jsx - phần 3/3

```
  return (
    <main className="panel">
      <h1>Đồng hồ bấm giờ</h1>
      <p className="clock">{formatElapsed(elapsed)}</p>
      <div className="row">
        <button onClick={start} disabled={isRunning}>Bắt đầu</button>
        <button onClick={stop} disabled={!isRunning}>Dừng</button>
        <button onClick={reset}>Đặt lại</button>
      </div>
      <p>Phút : giây : phần trăm giây</p>
    </main>
  );
}
```

src/index.css - thêm nếu chưa có .clock từ bài 17

```
.clock {
  font-size: clamp(32px, 8vw, 64px);
  font-family: monospace; font-variant-numeric: tabular-nums;
  text-align: center; margin: 20px 0;
}
```

Đọc định dạng phút : giây : phần trăm giây

61.230 ms tương ứng 1 phút, 1 giây và 23 phần trăm giây, nên hiện 01:01:23. Math.floor bỏ phần lẻ; % 60 lấy phần giây còn lại sau các phút; % 100 giữ phần trăm giây trong khoảng 0-99.

TỰ LÀM TRONG 5-10 PHÚT

Thêm dòng “Đang chạy” hoặc “Đã dừng” bên dưới.
Gợi ý: Dùng {isRunning ? "Đang chạy" : "Đã dừng"}. Không đọc runningRef.current trong JSX để quyết định dòng này.

Kiểm tra đồng hồ bấm giờ

Đây cũng là một bài ôn cách thiết kế state cho ứng dụng.

Thao tác	Kết quả cần thấy
Vừa mở trang	00:00:00; nút Dừng bị vô hiệu hóa.
Bắt đầu	Số tăng; nút Bắt đầu bị vô hiệu hóa.
Dừng sau vài giây	Số giữ nguyên, không tiếp tục nhảy.
Bắt đầu lần nữa	Tiếp tục từ số đã dừng, không về 0.
Đặt lại khi đang chạy	Về 00:00:00 và dừng hẳn.
Đặt lại khi đã dừng	Về 00:00:00; có thể bắt đầu từ đầu.
Rời demo	Cleanup hủy interval.

Vì sao không setElapsed(previous => previous + 10)?

Khoảng chờ 10 ms của interval không đảm bảo callback thật sự chạy chính xác mỗi 10 ms. Demo tính chênh lệch mốc thời gian để hạn chế sai lệch do callback bị trễ. Độ mượt hiển thị vẫn phụ thuộc trình duyệt và tab đang hoạt động.

> **GIỚI HẠN CỦA DEMO**
>
> Đây là đồng hồ trong một phiên trang web. Tải lại trang sẽ reset; không dùng nó như bộ đo bền vững qua đóng trình duyệt hoặc máy ngủ dài. Cách performance.now() tiến trong lúc hệ điều hành ngủ còn khác nhau giữa môi trường.

Tự trả lời trước khi kết thúc

- Vì sao elapsed là state nhưng startRef là ref?
- Vì sao Effect phụ thuộc isRunning mà không phụ thuộc elapsed?
- Nếu bỏ cleanup, chuyện gì có thể xảy ra khi bật/tắt hoặc unmount?

Gợi ý: elapsed phải hiện lên màn hình; startRef chỉ là mốc để tính. Nếu phụ thuộc elapsed, timer sẽ bị dựng lại theo mỗi lượt cập nhật. Cleanup giúp dừng timer không còn cần thiết.

Đọc thêm: R29, R24, R19, R27. Liên kết đầy đủ ở cuối tài liệu.

Trang trắng: kiểm tra theo thứ tự

Đừng sửa hàng loạt file trước khi biết lỗi nằm ở đâu.

Bước 1. Server có chạy đúng project không?

Terminal phải đứng trong thư mục có package.json của project đang học. Chạy npm run dev rồi mở đúng URL Local được in ra. Đừng mở trực tiếp index.html bằng đường dẫn file trên ổ D:. Nếu cổng 5173 bận, server có thể báo cổng khác.

Bước 2. Đọc lỗi trước khi đoán

Xem khung báo lỗi của Vite trên trang và Terminal. Sau đó bấm F12 > Console. Đọc dòng đỏ đầu tiên, tên file và số dòng. Một import lỗi có thể làm ứng dụng chưa chạy được từ đầu.

Bước 3. Thu nhỏ App để khoanh vùng

src/App.jsx - bản kiểm tra tạm thời

```
export default function App() {
  return <h1>React đã chạy được!</h1>;
}
```

Nếu dòng chữ hiện: bộ khung đang hoạt động. Nối lại đúng một demo rồi kiểm tra import, props và các biến trong demo đó. Nếu vẫn trắng: kiểm tra index.html, main.jsx, CSS và xem có đang mở nhầm URL/project hay không.

Kiểm tra	Mẫu đúng trong cuốn này
File tồn tại, đúng hoa/thường	src/ColorPicker.jsx khớp ./ColorPicker.jsx.
Có default export	export default function ColorPicker() { ... }
App trả component	return <ColorPicker />;
main import App	import App from "./App.jsx";
ID gốc khớp	HTML id="root" khớp getElementById("root").

KHI NHỜ NGƯỜI KHÁC SỬA

Gửi App.jsx, component đang mở và nguyên văn lỗi Console/Terminal. Chỉ ảnh trang trắng thường chưa đủ để biết nguyên nhân. Nếu nghi CSS, kiểm tra màu chữ, display: none và các import CSS cũ.

Đọc thông báo lỗi bằng tiếng Việt

Một thông báo đúng thường có ích hơn thử thay nhiều đoạn code.

Thông báo / biểu hiện	Nghĩa thường gặp và hướng sửa
Failed to resolve import	Không tìm thấy file/package. Kiểm tra tên, đường dẫn, thư mục, hoa/thường. Nếu package là prop-types của video cũ, demo React 19 trong cuốn này không cần import nó.
... is not defined	Biến/hàm chưa khai báo hoặc sai chữ hoa. Ví dụ PropTypes và propTypes là hai tên khác nhau; useState cũng phải được import.
does not provide an export named default	Import default nhưng file không export default, hoặc import nhầm file.
Too many re-renders	Hay do gọi setter ngay trong render hoặc viết onClick={handler()} thay vì truyền hàm.
Maximum update depth exceeded	Có thể có vòng cập nhật Effect/state. Xem setter có làm dependency thay đổi lặp lại không.
Objects are not valid as a React child	Bạn đang hiển thị cả object như {car}. Hãy hiển thị car.brand, car.model hoặc chuỗi đã định dạng.
Each child ... unique key	Thiếu key đúng ở phần tử ngoài cùng do map trả về. Đây thường là cảnh báo, không phải nguyên nhân duy nhất gây trang trắng.
Có value nhưng không gõ được	Thiếu onChange cập nhật state. Với checkbox, dùng checked và target.checked.
Danh sách không hiện, không lỗi rõ	Kiểm tra map callback có return JSX không; filter có đang lọc hết dữ liệu không.

ĐỪNG QUÊN NGUYÊN TẮC RETURN

Nếu xuống dòng ngay sau return rồi mới mở ngoặc, JavaScript có thể kết thúc lệnh return sớm. Hãy dùng `return (` trên cùng một dòng, rồi mới xuống dòng viết JSX.

Giữ project dễ học, dễ sửa

Những thao tác nhỏ giúp bạn tránh mất dấu khi chuyển qua nhiều bài.

Khi chuyển bài

1. Giữ main.jsx và CSS nền. Tạo đúng file mới trong src.

2. Thay App.jsx bằng bản của bài đang học. Một file chỉ có một export default.

3. Nếu code chia nhiều phần, nối theo thứ tự vào cùng một file rồi mới chạy.

4. CSS riêng luôn thêm ở cuối index.css, trừ bài 01 yêu cầu thay toàn bộ.

5. Bấm Ctrl+S; kiểm tra URL và thao tác theo phần kết quả cần thấy.

Ba lệnh hữu ích

Terminal - chạy trong thư mục project

```
npm ls react react-dom
npm run dev
npm run build
```

Lệnh đầu cho biết phiên bản đã cài. Lệnh dev mở môi trường phát triển. Lệnh build giúp phát hiện nhiều lỗi cú pháp/import khi đóng gói; build thành công không có nghĩa mọi tương tác đều đúng, nên vẫn cần bấm thử các nút.

Các giới hạn của bộ demo

• Dùng JavaScript và function component; không bắt bạn học TypeScript trước.

• Không có API, đăng nhập thật hoặc database. Đây là phần nền tảng frontend.

• Không tự lưu sau F5. Đừng hiểu mất dữ liệu khi tải lại là useState bị lỗi.

• App dùng ID ngẫu nhiên chạy trên localhost/HTTPS của trình duyệt hiện đại.

• Cách trình bày đơn giản; bạn có thể đổi CSS sau khi đã hiểu luồng dữ liệu.

NẾU LỖI XẢY RA NGAY SAU MỘT THAY ĐỔI

Quay lại thay đổi gần nhất và kiểm tra từng bước. Đừng đồng thời thay tên file, sửa logic, thêm thư viện và đổi CSS: bạn sẽ khó biết thay đổi nào gây lỗi.

Đọc thêm: R01, R08, R33. Liên kết đầy đủ ở cuối tài liệu.

Bảng nhớ nhanh React

Dùng để ôn lại, không cần học thuộc mọi dòng ngay hôm nay.

Bạn đang cần gì?	Chọn công cụ nào?	Ví dụ từ cuốn này
Chia giao diện thành phần nhỏ	Component	Card, Counter.
Đưa dữ liệu cha xuống con	Props	Student nhận name.
Hiện theo điều kiện	if, ? :, &&	UserGreeting.
Dữ liệu thành nhiều dòng	map + key ổn định	FruitList.
Đọc thao tác bấm/nhập	onClick, onChange	ClickDemo, ProfileForm.
Nhớ dữ liệu và cập nhật giao diện	useState	ColorPicker, TodoList.
Tính state từ state trước	Updater	previous => previous + 1.
Đổi object / array state	Tạo bản mới	Spread, map, filter.
Đồng bộ timer, window, bên ngoài	useEffect + cleanup	DigitalClock.
Nhiều tầng con đọc chung dữ liệu	Context + useContext	ContextDemo.
Nhớ giá trị không tự render / DOM	useRef	RefDemo, Stopwatch.

Năm câu tự nhắc khi đọc code

- Dữ liệu này đến từ props, state, ref hay được tính từ dữ liệu khác?
- Thao tác nào làm dữ liệu đổi? Handler hay Effect?
- Giá trị mới có phụ thuộc state trước không?
- Nếu sửa mảng/object, tôi đã tạo bản mới ở đúng tầng chưa?
- Nếu tạo timer/listener, tôi đã viết cleanup tương ứng chưa?

Ba bài luyện tổng hợp

Tự làm trước, xem hướng giải ở trang sau.

A. Danh bạ sinh viên: ôn props và danh sách

Tạo mảng students gồm ba object có id, name, major. Tạo StudentCard nhận student qua props. App dùng map để hiển thị ba thẻ. Thêm một input tìm kiếm theo tên, không phân biệt hoa/thường.

Tiêu chí	Tự thử
Dữ liệu gốc vẫn giữ nguyên	Xóa ô tìm kiếm thì đủ ba thẻ trở lại.
Key ổn định	Dùng id; không dùng tên mới tạo mỗi render.
Lọc đúng	Gõ một phần tên vẫn tìm thấy sinh viên phù hợp.

B. Giỏ hàng nhỏ: ôn array of objects

Tạo hai sản phẩm mẫu: Bút, giá 5.000 đ, số lượng 2; Vở, giá 12.000 đ, số lượng 1. Mỗi dòng có nút tăng/giảm số lượng; không cho xuống dưới 1. Hiện tổng tiền của toàn giỏ.

Kết quả ban đầu phải là 22.000 đ. Tăng số lượng Bút từ 2 lên 3 thì tổng thành 27.000 đ. Chỉ cập nhật object của sản phẩm được bấm; các sản phẩm khác không đổi.

C. Theme dùng chung: ôn Context

Tạo ThemeContext cung cấp theme và hàm đổi theme. Một component ở sâu có nút chuyển giữa light và dark. Component hiển thị khung chính đọc cùng context để đổi className tương ứng.

Kiểm tra: bấm nút ở sâu thì màu khung chính đổi; không cần truyền prop theme qua từng tầng. Màu nền tối phải có màu chữ đủ sáng để đọc được.

KHI NÀO NÊN CHUYỂN SANG HỌC API?

Khi bạn có thể tự giải thích và làm lại việc thêm/sửa/xóa trong bài 14-15 mà không nhìn toàn bộ mẫu. Khi đó việc nối React với backend Spring Boot sẽ dễ hiểu hơn vì bạn đã nắm phần giao diện và state.

Hướng giải ba bài luyện

Các đoạn dưới là phần cốt lõi để ghép vào bài tự làm, không phải App.jsx hoàn chỉnh.

A. Tìm kiếm: chỉ lưu chữ tìm kiếm trong state

Giả sử có mảng students và state keyword

```
const normalized = keyword.trim().toLowerCase();
const visibleStudents = students.filter((student) => (
  student.name.toLowerCase().includes(normalized)
));

// Trong JSX:
visibleStudents.map((student) => (
  <StudentCard key={student.id} student={student} />
));
```

Không ghi đè students bằng kết quả lọc. visibleStudents có thể tính lại mỗi render; khi keyword rỗng, includes("") đúng với mọi tên.

B. Đổi số lượng bằng id; tổng tiền là dữ liệu suy ra

Đặt hàm và phép tính trong component giỏ hàng

```
function changeQuantity(id, amount) {
  setProducts((previous) => previous.map((product) => (
    product.id === id
      ? { ...product, quantity: Math.max(1, product.quantity + amount) }
      : product
  )));
}

const total = products.reduce((sum, product) => (
  sum + product.price * product.quantity
), 0);
```

reduce ở đây cộng từng thành tiền vào sum, bắt đầu từ 0. Nút tăng gọi changeQuantity(id, 1); nút giảm truyền -1. Không cần tạo state riêng cho total.

C. Đổi theme bằng updater

Handler ở nơi giữ state theme

```
function toggleTheme() {
  setTheme((previous) => previous === "light" ? "dark" : "light");
}
```

Provider cung cấp { theme, toggleTheme }. Khung chính đọc theme để gán className; nút sâu đọc toggleTheme để gán onClick. Hai class CSS .light và .dark quyết định màu nền/chữ.

Nguồn đối chiếu: phần 1

Các liên kết có thể bấm được. Đối chiếu ngày 07/10/2026.

Thứ tự chủ đề và mốc video theo danh sách bạn cung cấp. Tài liệu dùng diễn giải và demo riêng, đối chiếu API với React, Vite và MDN; không tuyên bố chép lại toàn bộ nội dung video.

Video nền: React Full Course - Bro Code
https://www.youtube.com/watch?v=CgkZ7MvWUAA

[R01] Vite: bắt đầu và yêu cầu môi trường
https://vite.dev/guide/

[R02] React: component đầu tiên
https://react.dev/learn/your-first-component

[R03] React: quy tắc JSX
https://react.dev/learn/writing-markup-with-jsx

[R04] React: import và export component
https://react.dev/learn/importing-and-exporting-components

[R05] React: JavaScript, object style trong JSX
https://react.dev/learn/javascript-in-jsx-with-curly-braces

[R06] Vite: CSS, CSS Modules và các tính năng
https://vite.dev/guide/features.html

[R07] React: truyền props và giá trị mặc định
https://react.dev/learn/passing-props-to-a-component

[R08] React 19: thay đổi propTypes và defaultProps
https://react.dev/blog/2024/04/25/react-19-upgrade-guide

[R09] React: hiển thị có điều kiện
https://react.dev/learn/conditional-rendering

[R10] React: render danh sách và key
https://react.dev/learn/rendering-lists

[R11] React: xử lý sự kiện
https://react.dev/learn/responding-to-events

[R12] React: state là bộ nhớ của component
https://react.dev/learn/state-a-components-memory

[R13] React: tham chiếu useState
https://react.dev/reference/react/useState

[R14] React: state dưới dạng snapshot
https://react.dev/learn/state-as-a-snapshot

[R15] React DOM: input, value, checked, onChange
https://react.dev/reference/react-dom/components/input

[R16] React: hàng đợi cập nhật state
https://react.dev/learn/queueing-a-series-of-state-updates

[R17] React: cập nhật object trong state
https://react.dev/learn/updating-objects-in-state

Nguồn đối chiếu: phần 2

Mã Rxx ở cuối các bài dẫn tới các nguồn này.

[R18] React: cập nhật array trong state
https://react.dev/learn/updating-arrays-in-state

[R19] React: tham chiếu useEffect
https://react.dev/reference/react/useEffect

[R20] React: đồng bộ với Effects và cleanup
https://react.dev/learn/synchronizing-with-effects

[R21] React: khi không cần Effect
https://react.dev/learn/you-might-not-need-an-effect

[R22] React: hành vi kiểm tra của StrictMode
https://react.dev/reference/react/StrictMode

[R23] MDN: Date và các phương thức thời gian
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

[R24] MDN: setInterval và giới hạn timer
https://developer.mozilla.org/en-US/docs/Web/API/Window/setInterval

[R25] React: tham chiếu useContext
https://react.dev/reference/react/useContext

[R26] React: truyền dữ liệu qua nhiều tầng bằng context
https://react.dev/learn/passing-data-deeply-with-context

[R27] React: tham chiếu useRef
https://react.dev/reference/react/useRef

[R28] React: chọn state hay ref
https://react.dev/learn/referencing-values-with-refs

[R29] MDN: performance.now và đo chênh lệch thời gian
https://developer.mozilla.org/en-US/docs/Web/API/Performance/now

[R30] MDN: spread và sao chép nông
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

[R31] React DOM: createRoot
https://react.dev/reference/react-dom/client/createRoot

[R32] React: render và commit
https://react.dev/learn/render-and-commit

[R33] MDN: crypto.randomUUID
https://developer.mozilla.org/en-US/docs/Web/API/Crypto/randomUUID

[R34] Vite: file trong public và tài nguyên tĩnh
https://vite.dev/guide/assets.html

> **HỌC THEO BẢN NÀY**
>
> Ví dụ dùng function component và cách đặt mặc định bằng tham số phù hợp React 19. Khi tài liệu chính thức thay đổi, hãy xem ghi chú phiên bản của React/Vite đang cài trong project của bạn.